From Linear Models to Neural Networks

Linear & logistic regression, neurons, MLPs, universal approximation

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

Lecture 1 chose the loss; Lecture 2 expands the prediction rule

Keep from Lecture 1

Regression output $\rightarrow$ Gaussian model $\rightarrow$ MSE

Class probabilities $\rightarrow$ Bernoulli / categorical model $\rightarrow$ cross-entropy

Lecture 1 chose the loss; Lecture 2 expands the prediction rule

Keep from Lecture 1

Regression output $\rightarrow$ Gaussian model $\rightarrow$ MSE

Class probabilities $\rightarrow$ Bernoulli / categorical model $\rightarrow$ cross-entropy

Change in Lecture 2

Start with one weighted sum, expose what it cannot represent, then let hidden layers learn nonlinear features.

Lecture 1 chose the loss; Lecture 2 expands the prediction rule

Keep from Lecture 1

Regression output $\rightarrow$ Gaussian model $\rightarrow$ MSE

Class probabilities $\rightarrow$ Bernoulli / categorical model $\rightarrow$ cross-entropy

Change in Lecture 2

Start with one weighted sum, expose what it cannot represent, then let hidden layers learn nonlinear features.

$$\boldsymbol{x} \rightarrow f_{\boldsymbol{\theta}}(\boldsymbol{x}) \rightarrow \text{output parameters} \rightarrow \text{same Lecture 1 loss}$$

Lecture 1 chose the loss; Lecture 2 expands the prediction rule

Keep from Lecture 1

Regression output $\rightarrow$ Gaussian model $\rightarrow$ MSE

Class probabilities $\rightarrow$ Bernoulli / categorical model $\rightarrow$ cross-entropy

Change in Lecture 2

Start with one weighted sum, expose what it cannot represent, then let hidden layers learn nonlinear features.

$$\boldsymbol{x} \rightarrow f_{\boldsymbol{\theta}}(\boldsymbol{x}) \rightarrow \text{output parameters} \rightarrow \text{same Lecture 1 loss}$$

We will not re-derive the losses; today is about making f_θ more expressive.

The map for today

V VISUAL

One question drives the lecture: **what changes when one weighted sum becomes a network?**

The map for today

V VISUAL

One question drives the lecture: **what changes when one weighted sum becomes a network?**

The map for today

V VISUAL

One question drives the lecture: **what changes when one weighted sum becomes a network?**

weighted sum → expose the limitation → learn nonlinear features → compose them

The map for today

V VISUAL

One question drives the lecture: **what changes when one weighted sum becomes a network?**

weighted sum → expose the limitation → learn nonlinear features → compose them

We will calculate every stage for small numerical examples before writing the general matrix form.

One notation covers every prediction rule

Q QUESTION

Write $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ for the model's prediction at input $\boldsymbol{x}$.

$\boldsymbol{x}$: the input features
$\boldsymbol{\theta}$: all trainable weights and biases
$\boldsymbol{f_{\theta}}$: the rule that maps input to a score or prediction

One notation covers every prediction rule

Q QUESTION

Write $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ for the model's prediction at input $\boldsymbol{x}$.

$\boldsymbol{x}$: the input features
$\boldsymbol{\theta}$: all trainable weights and biases
$\boldsymbol{f_\theta}$: the rule that maps input to a score or prediction

Starting point — one weighted sum plus a bias:

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \boldsymbol{w}^\top \boldsymbol{x} + b$$

One notation covers every prediction rule

Q QUESTION

Write $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ for the model's prediction at input $\boldsymbol{x}$.

$\boldsymbol{x}$: the input features
$\boldsymbol{\theta}$: all trainable weights and biases
$\boldsymbol{f_{\theta}}$: the rule that maps input to a score or prediction

Starting point — one weighted sum plus a bias:

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \boldsymbol{w}^{\top}\boldsymbol{x} + b$$

We will build the network gradually; the full nested formula comes only after each part is concrete.

The linear starting point

Start with one weighted sum

For an input vector $\boldsymbol{x} \in \mathbb{R}^d$ and scalar target y,

$$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x} + b$$

Start with one weighted sum

For an input vector $\boldsymbol{x} \in \mathbb{R}^d$ and scalar target y,

$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x} + b$

For $\boldsymbol{x} = (2, -1)$, $\boldsymbol{w} = (1.5, 0.5)$, and $b = -0.2$:

$\hat{y} = 1.5(2) + 0.5(-1) - 0.2 = 2.3.$

Start with one weighted sum

For an input vector $\boldsymbol{x} \in \mathbb{R}^d$ and scalar target y,

$$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x} + b$$

For $\boldsymbol{x} = (2, -1)$, $\boldsymbol{w} = (1.5, 0.5)$, and $b = -0.2$:

$$\hat{y} = 1.5(2) + 0.5(-1) - 0.2 = 2.3.$$

Regression: read 2.3 as the predicted mean.

Classification: read the weighted sum as a score, then convert it to probabilities.

Start with one weighted sum

For an input vector $\boldsymbol{x} \in \mathbb{R}^d$ and scalar target y,

$$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x} + b$$

For $\boldsymbol{x} = (2, -1)$, $\boldsymbol{w} = (1.5, 0.5)$, and $b = -0.2$:

$$\hat{y} = 1.5(2) + 0.5(-1) - 0.2 = 2.3.$$

Regression: read 2.3 as the predicted mean.

Classification: read the weighted sum as a score, then convert it to probabilities.

The same weighted-sum core can feed different output models and losses from Lecture 1.

A whole batch is one matrix multiplication

For $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ and $\boldsymbol{y} \in \mathbb{R}^n$,

$$\hat{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{w} + b\mathbf{1} = \tilde{\boldsymbol{X}}\boldsymbol{\theta}.$$

A whole batch is one matrix multiplication

For $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ and $\boldsymbol{y} \in \mathbb{R}^{n}$,

$$\hat{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{w} + b\mathbf{1} = \tilde{\boldsymbol{X}}\boldsymbol{\theta}.$$

$$\tilde{\boldsymbol{X}} \in \mathbb{R}^{n \times (d+1)}, \quad \boldsymbol{\theta} \in \mathbb{R}^{d+1}, \quad \hat{\boldsymbol{y}} \in \mathbb{R}^{n}.$$

A whole batch is one matrix multiplication

For $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ and $\boldsymbol{y} \in \mathbb{R}^{n}$,

$$\hat{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{w} + b\mathbf{1} = \tilde{\boldsymbol{X}}\boldsymbol{\theta}.$$

$$\tilde{\boldsymbol{X}} \in \mathbb{R}^{n \times (d+1)}, \quad \boldsymbol{\theta} \in \mathbb{R}^{d+1}, \quad \hat{\boldsymbol{y}} \in \mathbb{R}^{n}.$$

Rows are examples; columns are features; matrix multiplication performs all n predictions together.

For regression, one weighted sum gives one flat trend

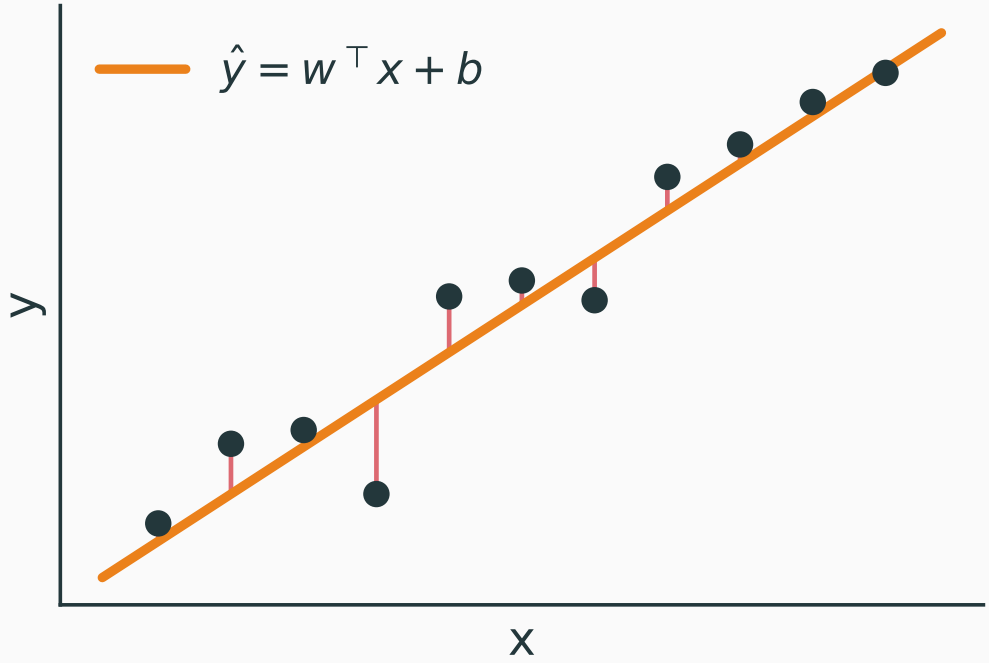
V VISUAL

$$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x} + b$$

For regression, one weighted sum gives one flat trend

V VISUAL

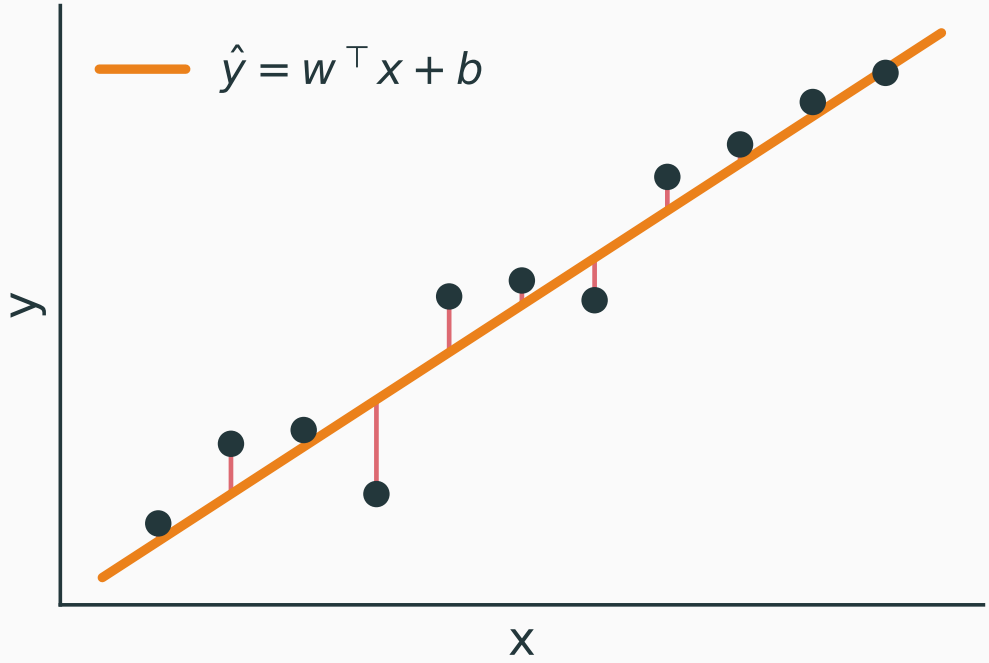
$$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x} + b$$



For regression, one weighted sum gives one flat trend

V VISUAL

$$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x} + b$$



The model can tilt and shift a line or plane, but it cannot bend to follow nonlinear structure.

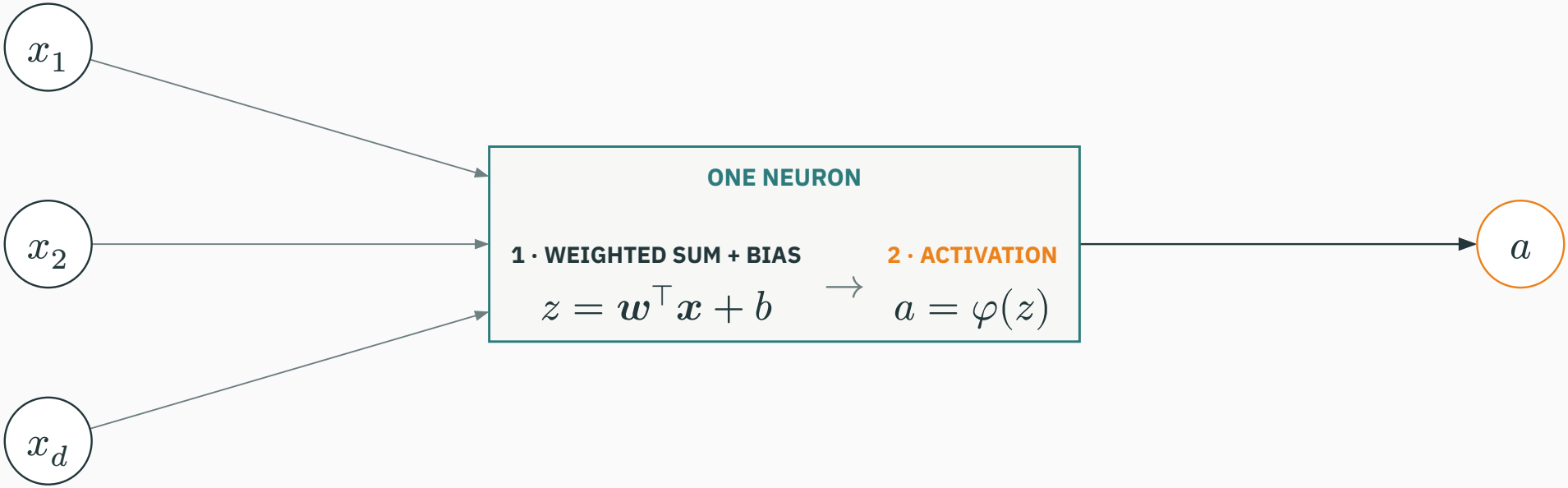
From a linear score to a neuron

A neuron has two stages

First compute a weighted sum plus bias; then pass that value through an activation.

A neuron has two stages

First compute a weighted sum plus bias; then pass that value through an activation.



One neuron recovers familiar Lecture 1 models

Let $z = \boldsymbol{w}^\top \boldsymbol{x} + b$. The output interpretation chooses the last step:

Regression

identity output

$\hat{y} = z$

One neuron recovers familiar Lecture 1 models

Let $z = \boldsymbol{w}^\top \boldsymbol{x} + b$. The output interpretation chooses the last step:

Regression

identity output

$\hat{y} = z$

Binary class

sigmoid output

$p = \sigma(z)$

One neuron recovers familiar Lecture 1 models

Let $z = \boldsymbol{w}^\top \boldsymbol{x} + b$. The output interpretation chooses the last step:

Regression

identity output

$\hat{y} = z$

Binary class

sigmoid output

$p = \sigma(z)$

K classes

K scores + softmax

$\boldsymbol{p} = \mathrm{softmax}(\boldsymbol{z})$

One neuron recovers familiar Lecture 1 models

Let $z = \boldsymbol{w}^\top \boldsymbol{x} + b$. The output interpretation chooses the last step:

Regression
identity output
$\hat{y} = z$

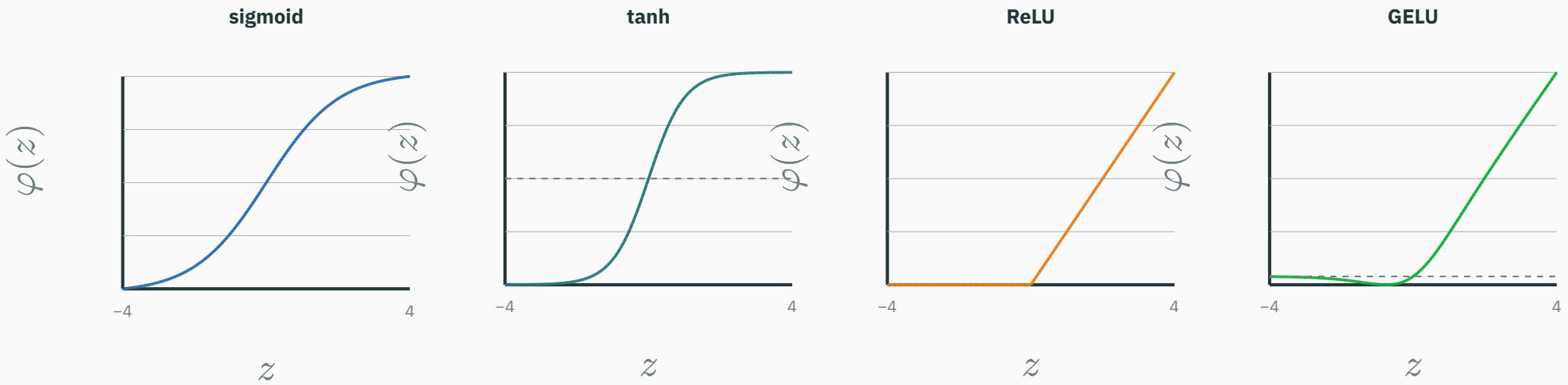
Binary class
sigmoid output
$p = \sigma(z)$

K classes
K scores + softmax
$\boldsymbol{p} = \mathrm{softmax}(\boldsymbol{z})$

Lecture 1 tells us how to interpret the output and choose the loss; Lecture 2 changes the features entering that output.

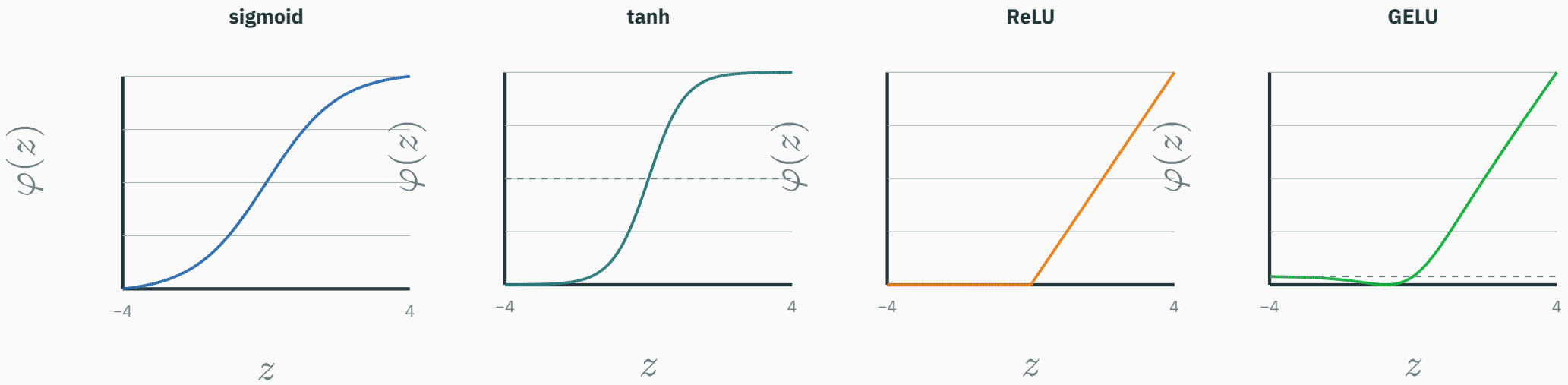
Common activation functions

V VISUAL



Common activation functions

V VISUAL



sigmoid → gates/probabilities · tanh → zero-centred · **ReLU** → common hidden unit · GELU → Transformers

Question: do more linear layers create new shapes?

Q QUESTION

Remove every activation. What can a stack of “weighted sum + bias” layers represent?

A. One equivalent weighted sum + bias

B. Any curved decision boundary

C. Only a constant function

D. A probability distribution automatically

Answer: line after line is still a line

A ANSWER

Algebraic view

$$h_1 = W_1 x + b_1$$

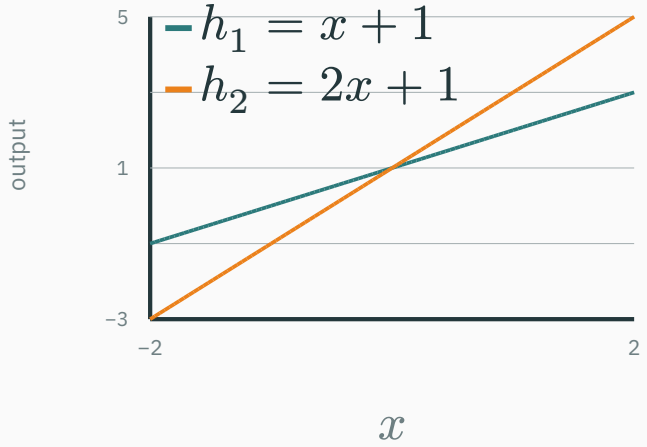
$$h_2 = W_2 h_1 + b_2$$

$$h_2 = \underbrace{W_2 W_1}_{\text{one matrix}} x + \underbrace{(W_2 b_1 + b_2)}_{\text{one bias}}$$

$W_2 W_1$ is one new weight matrix; the remaining term is one new bias vector.

Geometric view

$$h_1 = x + 1, \quad h_2 = 2h_1 - 1 = 2x + 1$$



The slope changes; no bend appears.

Correct: A. Without a nonlinear activation, extra layers still reduce to one weighted sum plus bias.

Interactive: activation functions

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/dl-teaching/interactives/activation-explorer.html

Compare activations and their **derivatives**, and see where gradients vanish.
Controls: activation type · input z · show derivative · compare saturation.

Interactive: activation functions

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/dl-teaching/interactives/activation-explorer.html

Compare activations and their **derivatives**, and see where gradients vanish.
Controls: activation type · input z · show derivative · compare saturation.

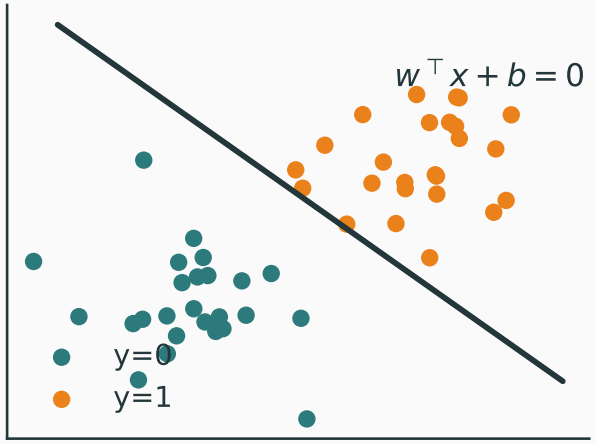
Explore large positive and negative logits, where sigmoid's derivative becomes tiny.

Where a single weighted sum fails

Single-layer classifiers draw only straight boundaries

V VISUAL

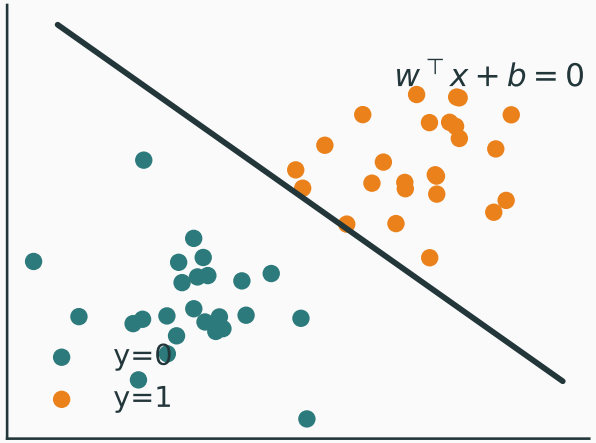
Binary: threshold one score $z = \boldsymbol{w}^\top \boldsymbol{x} + b$.



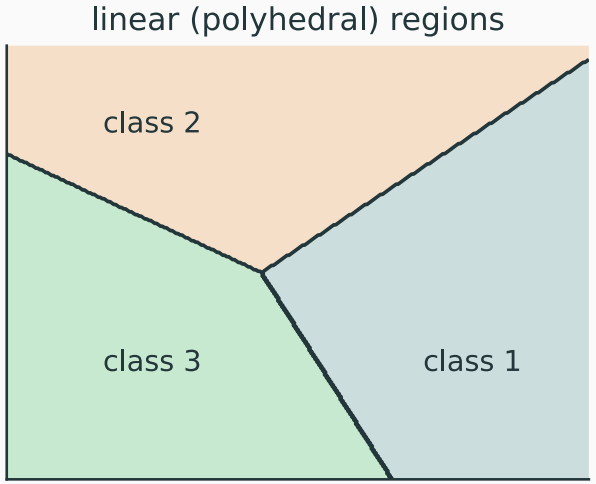
Single-layer classifiers draw only straight boundaries

V VISUAL

Binary: threshold one score $z = \boldsymbol{w}^\top \boldsymbol{x} + b$.



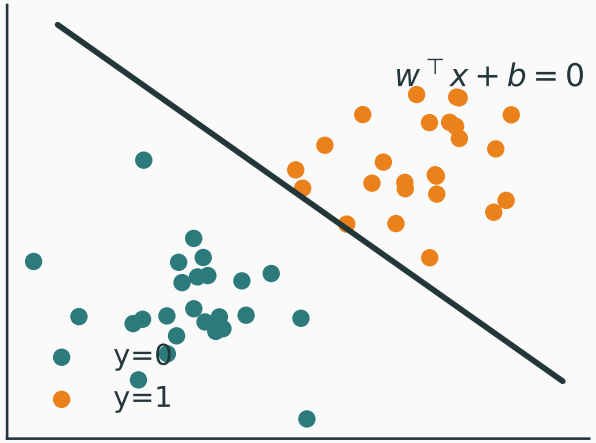
Multi-class: compare several linear scores.



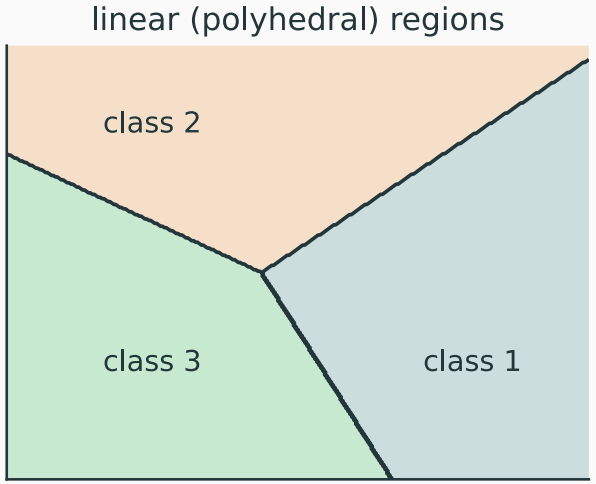
Single-layer classifiers draw only straight boundaries

V VISUAL

Binary: threshold one score $z = \boldsymbol{w}^\top \boldsymbol{x} + b$.



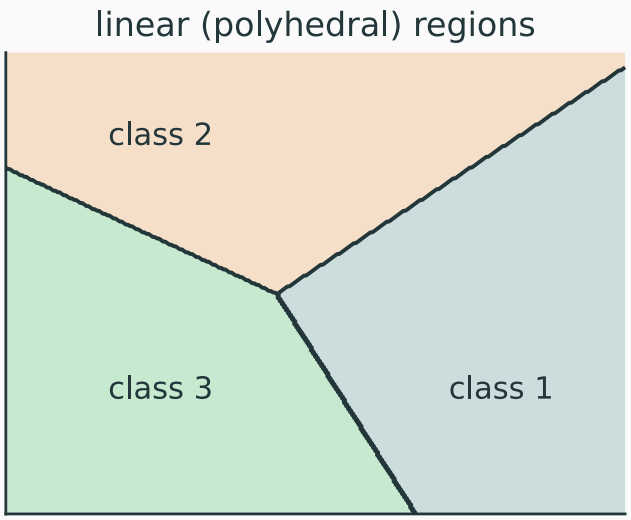
Multi-class: compare several linear scores.



Sigmoid and softmax change scores into probabilities; they do not bend the boundaries in input space.

Question: why are the multi-class boundaries straight?

Q QUESTION



three linear scores divide the input plane

The classifier predicts

$$\hat{y}(\boldsymbol{x}) = \arg\max_k p_{k(\boldsymbol{x})}.$$

When is a point on the boundary between predicted classes i and j?

A. Whenever $p_i = p_j$
B. When $p_i = p_j$ and both are maximal
C. Whenever $p_i + p_j = 1$
D. Only when all K probabilities are equal

Answer, part 1: equal probabilities give a flat tie set

A ANSWER

Softmax preserves score comparisons.

$$\frac{p_i}{p_j} = \frac{\exp(z_i)}{\exp(z_j)} = \exp(z_i - z_j).$$

Therefore

$$p_i = p_j \leftrightarrow z_i = z_j,$$

$$p_i > p_j \leftrightarrow z_i > z_j.$$

$\arg\max_k p_k = \arg\max_k z_k$: softmax changes the scale, not the ranking.

Linear-score ties are flat.

$$z_{k(\boldsymbol{x})} = \boldsymbol{w}_k^\top \boldsymbol{x} + b_k$$

$$z_i = z_j \leftrightarrow (\boldsymbol{w}_i - \boldsymbol{w}_j)^\top \boldsymbol{x} + (b_i - b_j) = 0.$$

This equation is a line in 2-D and a hyperplane in higher dimensions.

It is the **candidate pairwise boundary** between classes i and j.

Pairwise equality explains why every candidate boundary is straight. But is every tie visible?

Answer, part 2: the tied classes must also be on top

A ANSWER

A pairwise tie alone is not enough.

Suppose

$\boldsymbol{z} = (1, 1, 3).$

Then

$p_1 = p_2 < p_3.$

The prediction is class 3. The point lies on $z_1 = z_2$, but **not** on a visible class-1/class-2 boundary.

Decision region for class i

$$\mathcal{R}_i = \{\boldsymbol{x} : z_i \geq z_l \quad \text{for every} \quad l\}.$$

Boundary shared by classes i and j

$$\mathcal{B}_{ij} = \mathcal{R}_i \cap \mathcal{R}_j$$

$$\mathcal{B}_{ij} = \{\boldsymbol{x} : z_i = z_j \geq z_l \quad \text{for every} \quad l\}.$$

Equivalently, $p_i = p_j$ and both probabilities are maximal.

Correct: B. The visible boundary is the top-scoring portion of a pairwise linear tie set.

XOR exposes the missing ingredient

V VISUAL

No single line separates the two XOR classes.

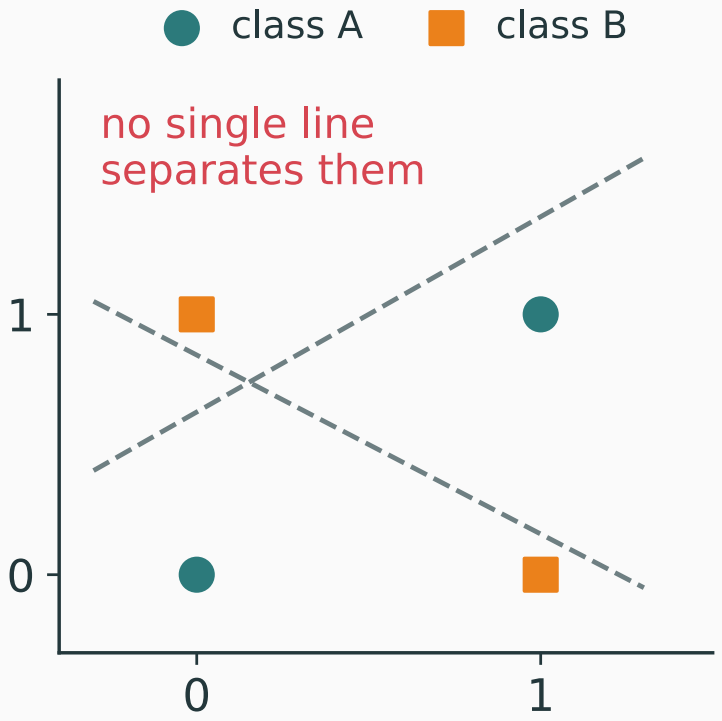
The model first needs a transformation of the input—new features in which the classes become separable.

XOR exposes the missing ingredient

V VISUAL

No single line separates the two XOR classes.

The model first needs a transformation of the input—new features in which the classes become separable.

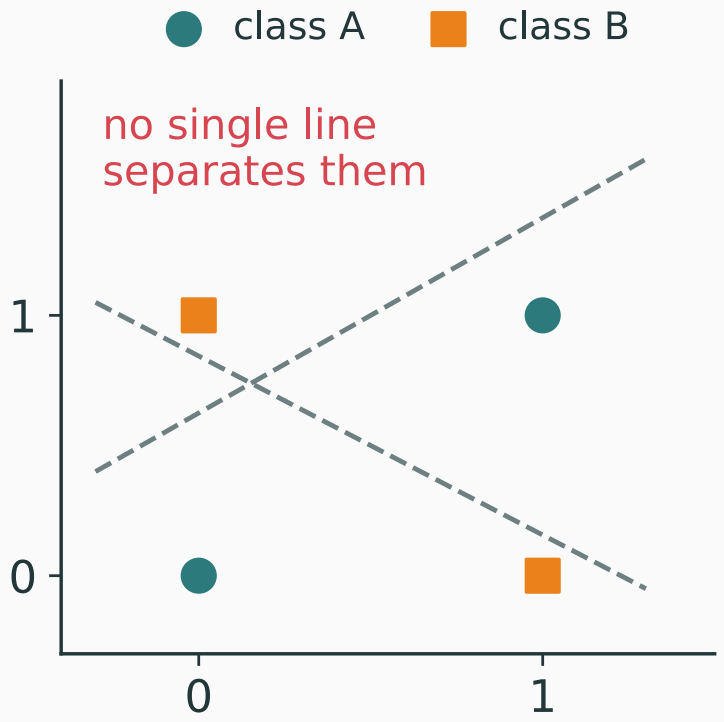


XOR exposes the missing ingredient

V VISUAL

No single line separates the two XOR classes.

The model first needs a transformation of the input—new features in which the classes become separable.



Hidden nonlinear units learn the transformation instead of asking us to hand-design it.

The multilayer perceptron

A hidden layer learns features before the output

Instead of $\boldsymbol{z} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$, insert a nonlinear layer first:

$$\boldsymbol{h} = \varphi(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1), \quad \boldsymbol{z} = \boldsymbol{W}_2\boldsymbol{h} + \boldsymbol{b}_2$$

A hidden layer learns features before the output

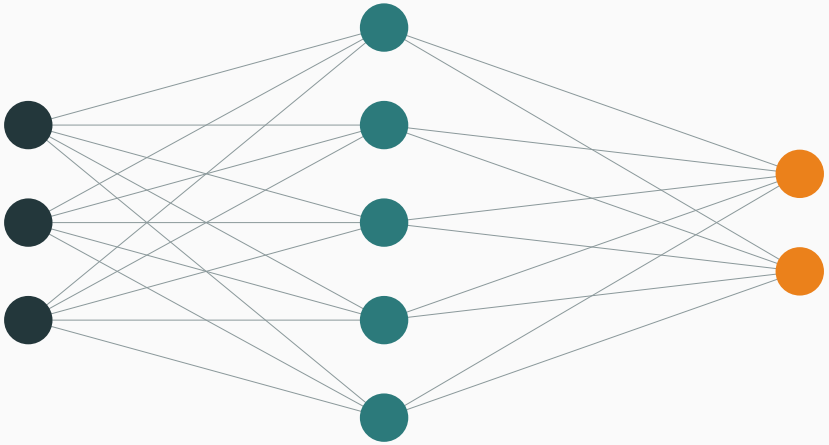
Instead of $\boldsymbol{z} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$, insert a nonlinear layer first:

$$\boldsymbol{h} = \varphi(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1), \quad \boldsymbol{z} = \boldsymbol{W}_2\boldsymbol{h} + \boldsymbol{b}_2$$

For classification, $\boldsymbol{p} = \mathrm{softmax}(\boldsymbol{z})$. This is a **one-hidden-layer MLP**.

Layer dimensions must agree

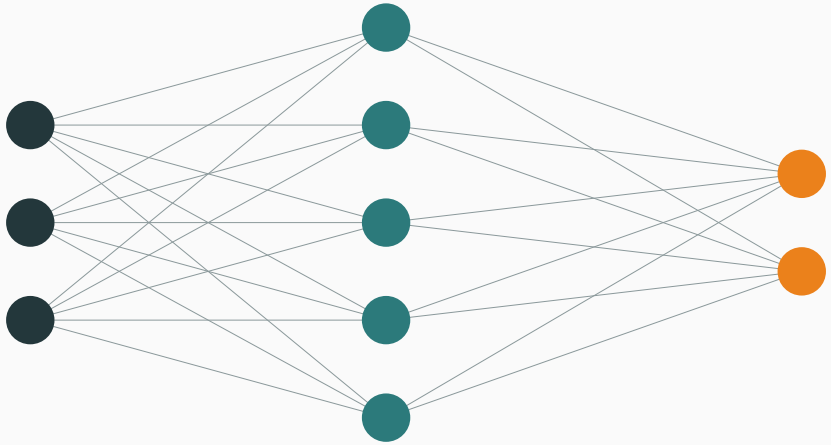
V VISUAL



d **inputs** m **hidden units** K **output scores**

Layer dimensions must agree

V VISUAL

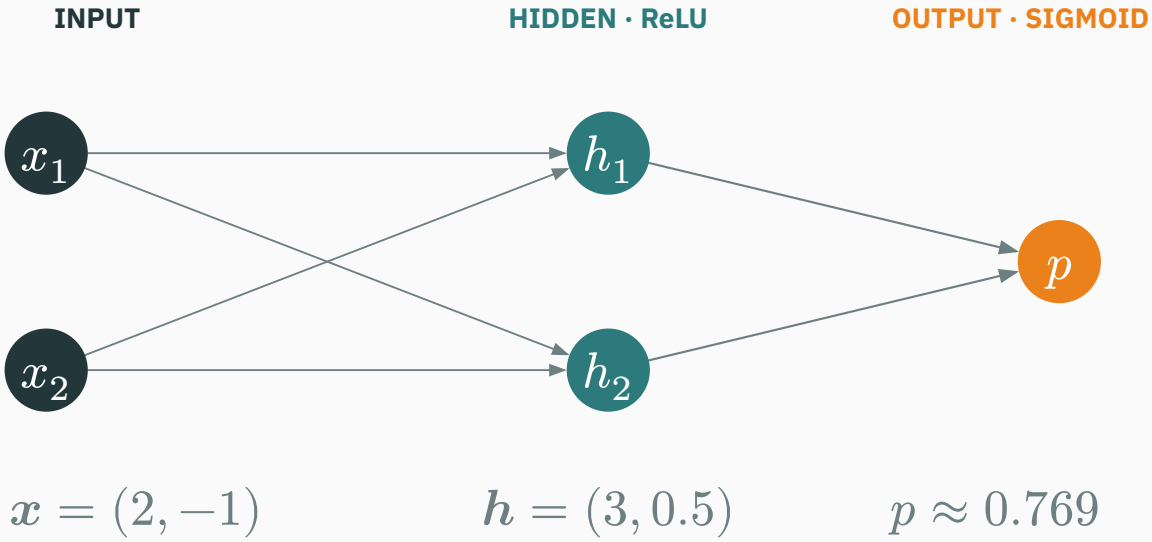


d **inputs** m **hidden units** K **output scores**

$$\boldsymbol{x} \in \mathbb{R}^d$$
$$\boldsymbol{W}_1 \in \mathbb{R}^{m \times d}, \quad \boldsymbol{b}_1 \in \mathbb{R}^m$$
$$\boldsymbol{h} \in \mathbb{R}^m$$
$$\boldsymbol{W}_2 \in \mathbb{R}^{K \times m}, \quad \boldsymbol{b}_2 \in \mathbb{R}^K$$
$$\boldsymbol{z} \in \mathbb{R}^K$$

First see the complete $2 \to 2 \to 1$ network

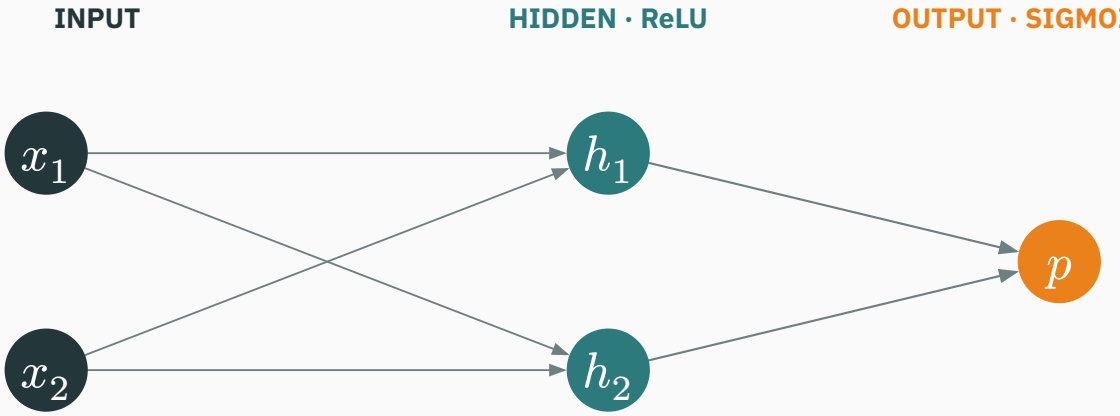
V VISUAL



$x = (2, -1)$ $h = (3, 0.5)$ $p \approx 0.769$

First see the complete $2 \to 2 \to 1$ network

V VISUAL



$x = (2, -1)$ $\quad$ $h = (3, 0.5)$ $\quad$ $p \approx 0.769$

Hidden layer

$$\boldsymbol{W}_1 = \begin{pmatrix} 1 & -1 \\ 0.5 & 1 \end{pmatrix}, \quad \boldsymbol{b}_1 = (0, 0.5)$$

Output layer

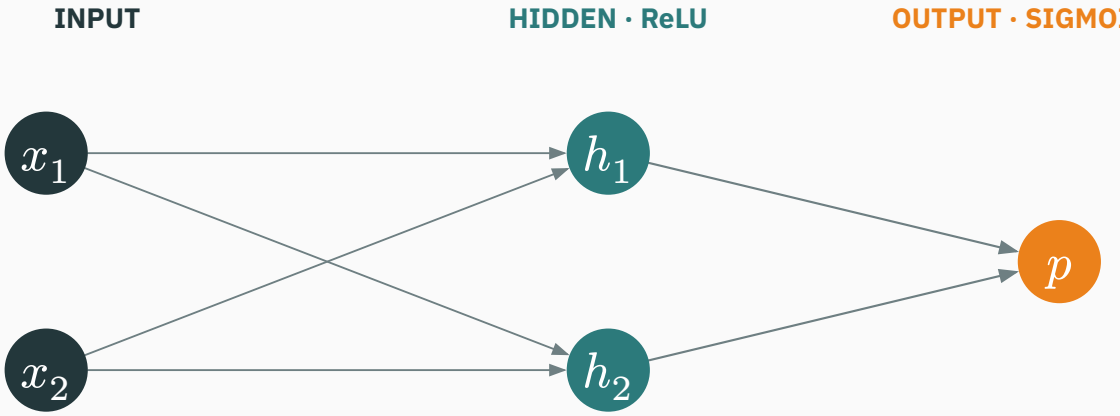
$$\boldsymbol{w}_2 = (0.8, -0.4), \quad b_2 = -1$$

Training example

$$y = 1, \quad \mathcal{L} = -\log p$$

First see the complete $2 \to 2 \to 1$ network

V VISUAL



$x = (2, -1)$ $h = (3, 0.5)$ $p \approx 0.769$

Hidden layer

$$\boldsymbol{W}_1 = \begin{pmatrix} 1 & -1 \\ 0.5 & 1 \end{pmatrix}, \quad \boldsymbol{b}_1 = (0, 0.5)$$

Output layer

$$\boldsymbol{w}_2 = (0.8, -0.4), \quad b_2 = -1$$

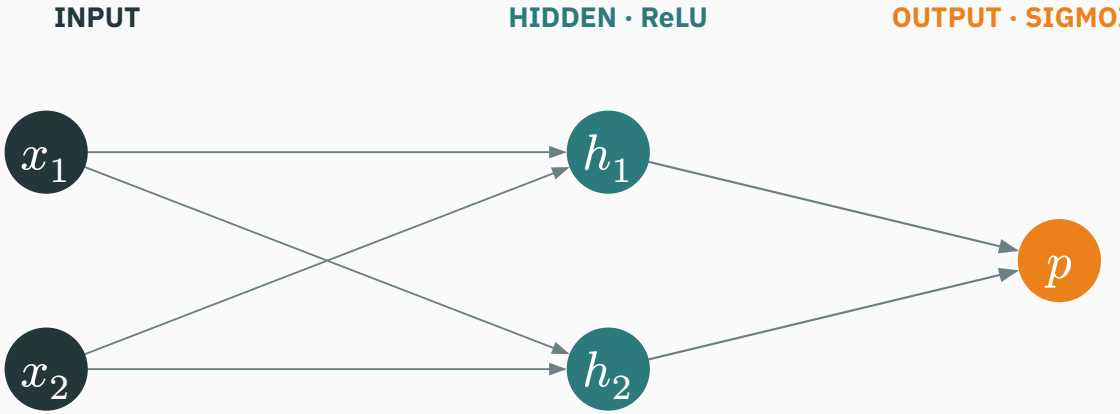
Training example

$$y = 1, \quad \mathcal{L} = -\log p$$

$$\boldsymbol{x} \to \boldsymbol{u} = \boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1 \to \boldsymbol{h} = \mathrm{ReLU}(\boldsymbol{u}) \to z = \boldsymbol{w}_2^\top \boldsymbol{h} + b_2 \to p = \sigma(z) \to \mathcal{L}$$

First see the complete $2 \to 2 \to 1$ network

V VISUAL



$x = (2, -1)$ $h = (3, 0.5)$ $p \approx 0.769$

Hidden layer

$$\boldsymbol{W}_1 = \begin{pmatrix} 1 & -1 \\ 0.5 & 1 \end{pmatrix}, \quad \boldsymbol{b}_1 = (0, 0.5)$$

Output layer

$$\boldsymbol{w}_2 = (0.8, -0.4), \quad b_2 = -1$$

Training example

$$y = 1, \quad \mathcal{L} = -\log p$$

$$\boldsymbol{x} \to \boldsymbol{u} = \boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1 \to \boldsymbol{h} = \mathrm{ReLU}(\boldsymbol{u}) \to z = \boldsymbol{w}_2^\top \boldsymbol{h} + b_2 \to p = \sigma(z) \to \mathcal{L}$$

Now calculate the same network from left to right.

Work a $2 \to 2$ hidden layer by hand

D DERIVATION

Let $\boldsymbol{x} = (2, -1)$ and

$$\boldsymbol{W}_1 = \begin{pmatrix} 1 & -1 \\ 0.5 & 1 \end{pmatrix}, \quad \boldsymbol{b}_1 = (0, 0.5).$$

Work a $2 \to 2$ hidden layer by hand

D DERIVATION

Let $\boldsymbol{x} = (2, -1)$ and

$$\boldsymbol{W}_1 = \begin{pmatrix} 1 & -1 \\ 0.5 & 1 \end{pmatrix}, \quad \boldsymbol{b}_1 = (0, 0.5).$$

The hidden pre-activation is

$$\boldsymbol{u} = \boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1 = (3, 0.5).$$

Work a $2 \to 2$ hidden layer by hand

D DERIVATION

Let $\boldsymbol{x} = (2, -1)$ and

$$\boldsymbol{W}_1 = \begin{pmatrix} 1 & -1 \\ 0.5 & 1 \end{pmatrix}, \quad \boldsymbol{b}_1 = (0, 0.5).$$

The hidden pre-activation is

$$\boldsymbol{u} = \boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1 = (3, 0.5).$$

Apply ReLU elementwise:

$$\boldsymbol{h} = \mathrm{ReLU}(\boldsymbol{u}) = (3, 0.5).$$

Work a $2 \to 2$ hidden layer by hand

D DERIVATION

Let $\boldsymbol{x} = (2, -1)$ and

$$\boldsymbol{W}_1 = \begin{pmatrix} 1 & -1 \\ 0.5 & 1 \end{pmatrix}, \quad \boldsymbol{b}_1 = (0, 0.5).$$

The hidden pre-activation is

$$\boldsymbol{u} = \boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1 = (3, 0.5).$$

Apply ReLU elementwise:

$$\boldsymbol{h} = \mathrm{ReLU}(\boldsymbol{u}) = (3, 0.5).$$

The hidden layer converts the raw input into two learned features.

The output layer turns hidden features into a prediction

D DERIVATION

For binary classification, choose $\boldsymbol{w}_2 = (0.8, -0.4)$ and $b_2 = -1$.

The output layer turns hidden features into a prediction

D DERIVATION

For binary classification, choose $\boldsymbol{w}_2 = (0.8, -0.4)$ and $b_2 = -1$.

$$z = \boldsymbol{w}_2^\top \boldsymbol{h} + b_2 = 0.8(3) - 0.4(0.5) - 1 = 1.2.$$

The output layer turns hidden features into a prediction

D DERIVATION

For binary classification, choose $\boldsymbol{w}_2 = (0.8, -0.4)$ and $b_2 = -1$.

$$z = \boldsymbol{w}_2^\top \boldsymbol{h} + b_2 = 0.8(3) - 0.4(0.5) - 1 = 1.2.$$

$$p = \sigma(1.2) \approx 0.769.$$

The output layer turns hidden features into a prediction

D DERIVATION

For binary classification, choose $\boldsymbol{w}_2 = (0.8, -0.4)$ and $b_2 = -1$.

$$z = \boldsymbol{w}_2^\top \boldsymbol{h} + b_2 = 0.8(3) - 0.4(0.5) - 1 = 1.2.$$

$$p = \sigma(1.2) \approx 0.769.$$

If $y = 1$, then $\mathcal{L} = -\log(0.769) \approx 0.262$.

The output layer turns hidden features into a prediction

D DERIVATION

For binary classification, choose $\boldsymbol{w}_2 = (0.8, -0.4)$ and $b_2 = -1$.

$$z = \boldsymbol{w}_2^\top \boldsymbol{h} + b_2 = 0.8(3) - 0.4(0.5) - 1 = 1.2.$$

$$p = \sigma(1.2) \approx 0.769.$$

If $y = 1$, then $\mathcal{L} = -\log(0.769) \approx 0.262$.

$x \to \boldsymbol{h} \to z \to p \to \mathcal{L}$ **is one complete forward pass.**

Each hidden neuron creates one learned feature

$$h_j = \varphi\left(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j\right)$$

Each hidden neuron creates one learned feature

$$h_j = \varphi\left(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j\right)$$

- the first layer **learns features**;
- the output layer **combines** them.

Each hidden neuron creates one learned feature

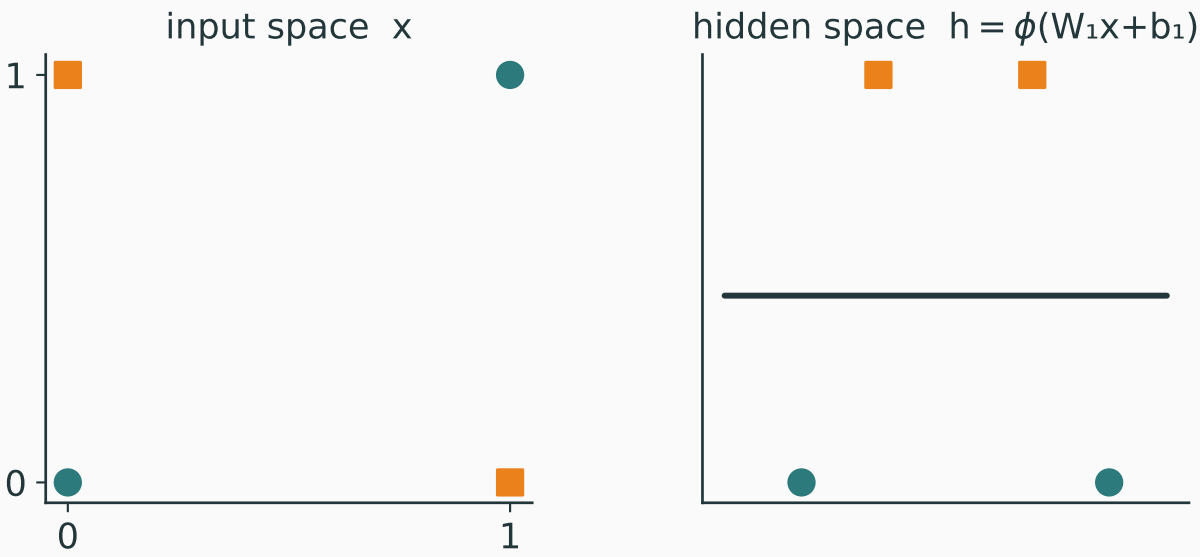
$$h_j = \varphi\left(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j\right)$$

- the first layer **learns features**;
- the output layer **combines** them.

For ReLU, $h_j = \max\left(0, \boldsymbol{w}_j^\top \boldsymbol{x} + b_j\right)$ — each unit is active on one side of a hyperplane.

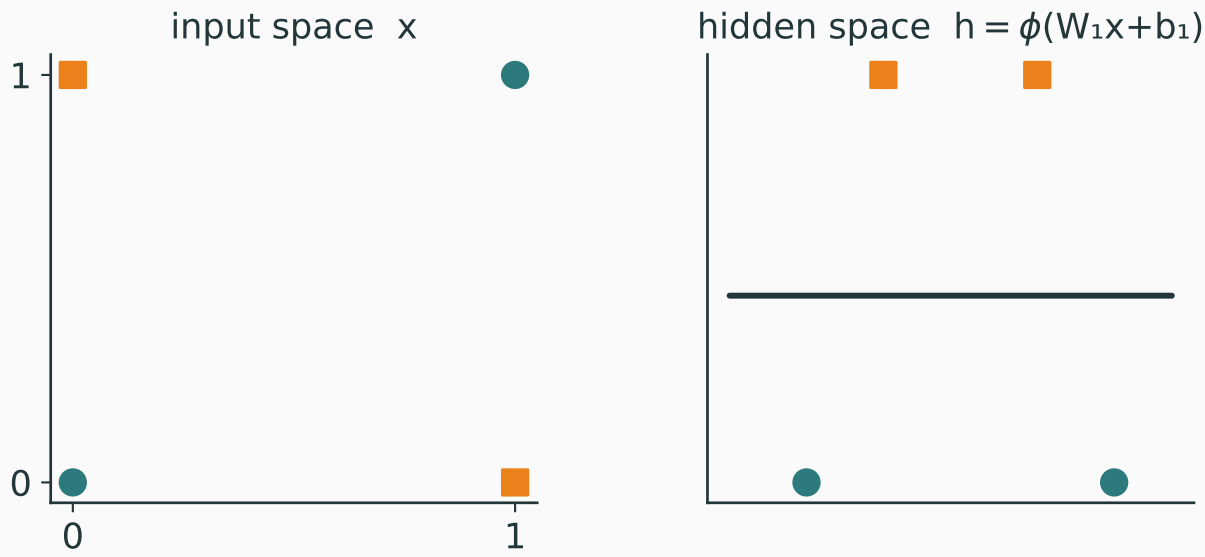
The feature-transformation view

V VISUAL



The feature-transformation view

V VISUAL

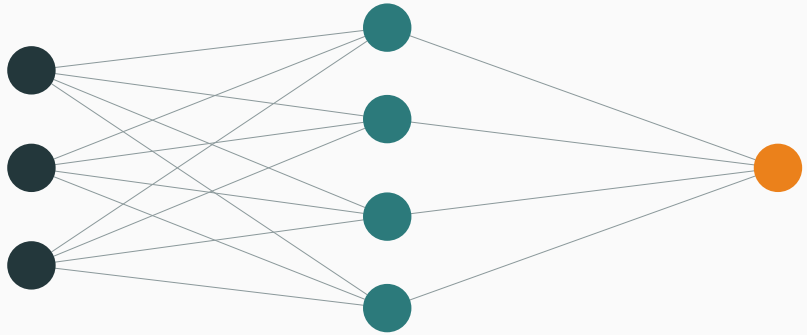


An MLP can learn a representation where classification becomes linear.

Binary uses one output node; K-class uses K

V VISUAL

BINARY CLASSIFICATION



d inputs

m hidden units

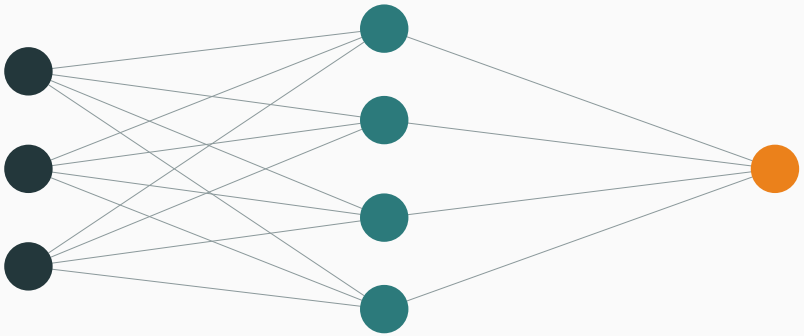
1 output node

$$z \in \mathbb{R} \rightarrow p = \sigma(z)$$

Binary uses one output node; K-class uses K

V VISUAL

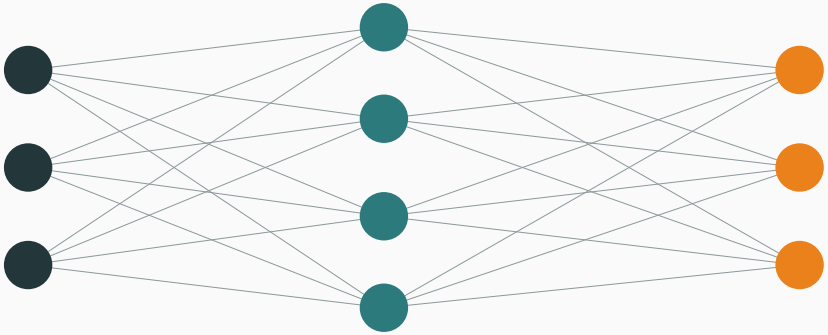
BINARY CLASSIFICATION



d inputs m hidden units 1 output node

$$z \in \mathbb{R} \to p = \sigma(z)$$

MULTI-CLASS CLASSIFICATION



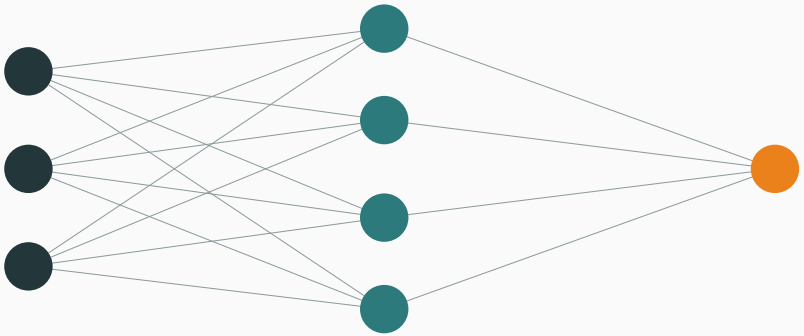
d inputs m hidden units K output nodes

$$\boldsymbol{z} \in \mathbb{R}^K \to \boldsymbol{p} = \text{softmax}(\boldsymbol{z})$$

Binary uses one output node; K-class uses K

V VISUAL

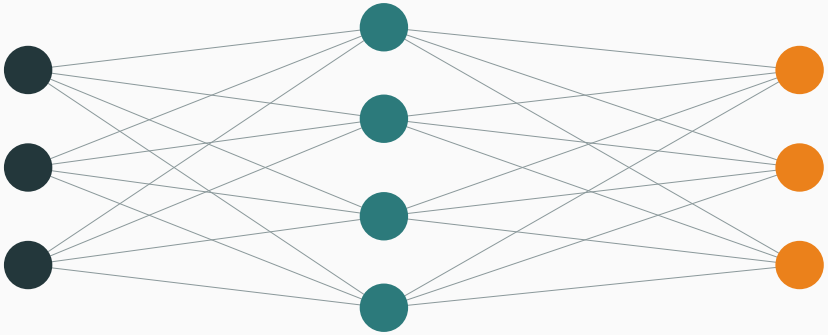
BINARY CLASSIFICATION



d inputs m hidden units 1 output node

$$z \in \mathbb{R} \to p = \sigma(z)$$

MULTI-CLASS CLASSIFICATION



d inputs m hidden units K output nodes

$$\boldsymbol{z} \in \mathbb{R}^K \to \boldsymbol{p} = \mathrm{softmax}(\boldsymbol{z})$$

Binary: 1 node gives p and $1-p$. Multi-class: K nodes give K softmax probabilities.

Depth stacks several hidden layers

V VISUAL

$$\boldsymbol{h}^{(1)} = \varphi(\boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1)$$

$$\boldsymbol{h}^{(\ell)} = \varphi\big(\boldsymbol{W}_\ell \boldsymbol{h}^{(\ell-1)} + \boldsymbol{b}_\ell\big), \quad \ell = 2, \ldots, L-1$$

$$\boldsymbol{z} = \boldsymbol{W}_L \boldsymbol{h}^{(L-1)} + \boldsymbol{b}_L$$

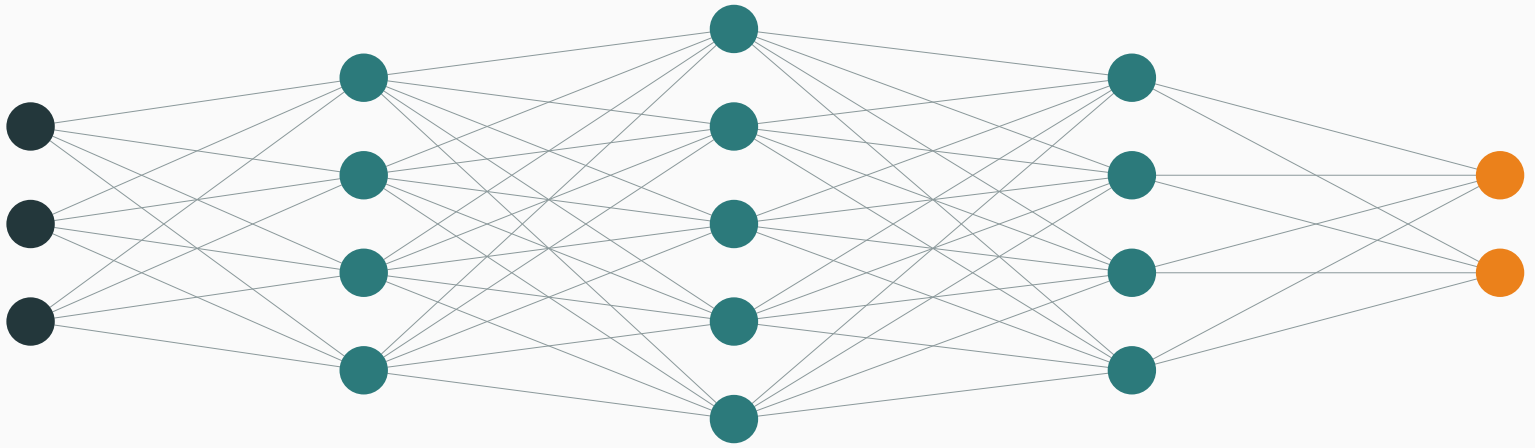
Depth stacks several hidden layers

V VISUAL

$$\boldsymbol{h}^{(1)} = \varphi(\boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1)$$

$$\boldsymbol{h}^{(\ell)} = \varphi\big(\boldsymbol{W}_\ell \boldsymbol{h}^{(\ell-1)} + \boldsymbol{b}_\ell\big), \quad \ell = 2, \ldots, L-1$$

$$\boldsymbol{z} = \boldsymbol{W}_L \boldsymbol{h}^{(L-1)} + \boldsymbol{b}_L$$



input $\boldsymbol{x}$ hidden $\boldsymbol{h}^{(1)}$ hidden $\boldsymbol{h}^{(2)}$ hidden $\boldsymbol{h}^{(L-1)}$ output $\boldsymbol{z}$

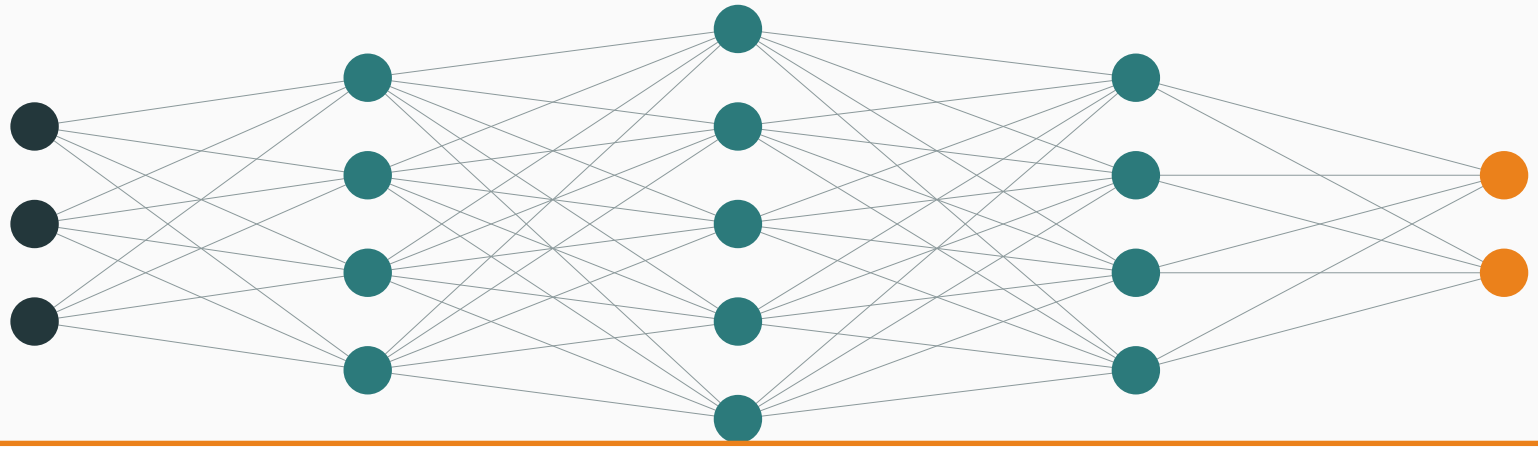
Depth stacks several hidden layers

V VISUAL

$$h^{(1)} = \varphi(W_1 x + b_1)$$

$$h^{(\ell)} = \varphi\big(W_\ell h^{(\ell-1)} + b_\ell\big), \quad \ell = 2, \ldots, L-1$$

$$z = W_L h^{(L-1)} + b_L$$



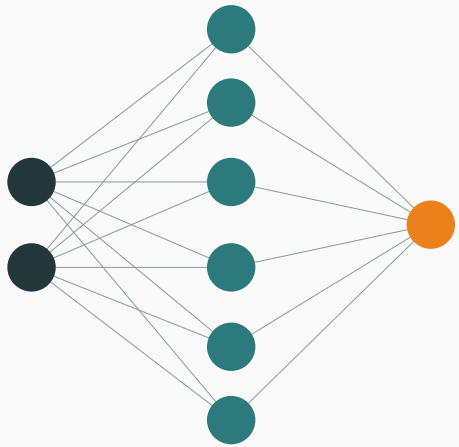
Each hidden layer transforms the previous representation. The output layer then produces task-specific scores or predictions.

Width and depth enlarge a network differently

V VISUAL

WIDER

One hidden layer, **six neurons**

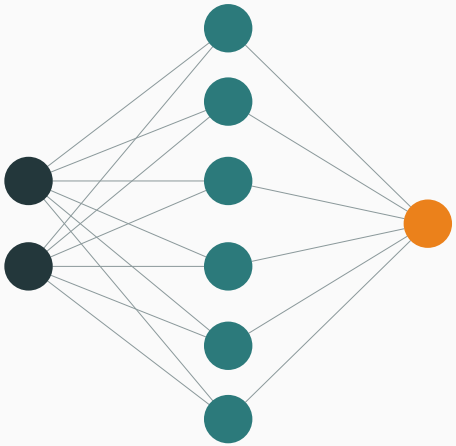


Width and depth enlarge a network differently

V VISUAL

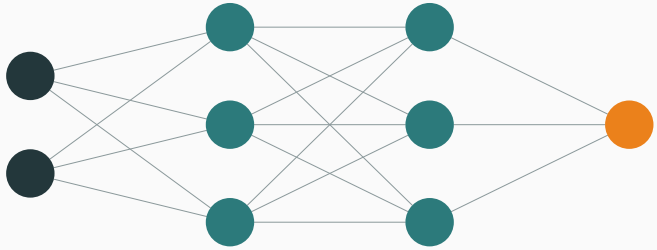
WIDER

One hidden layer, **six neurons**



DEEPER

Two hidden layers, **three neurons each**

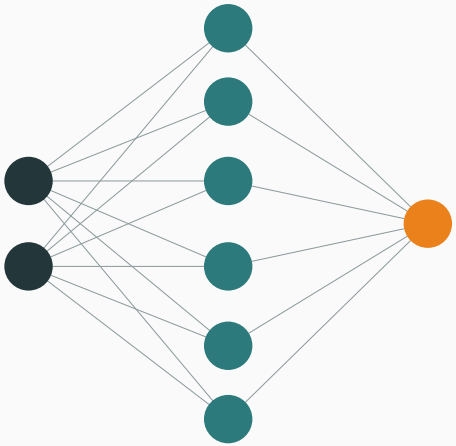


Width and depth enlarge a network differently

V VISUAL

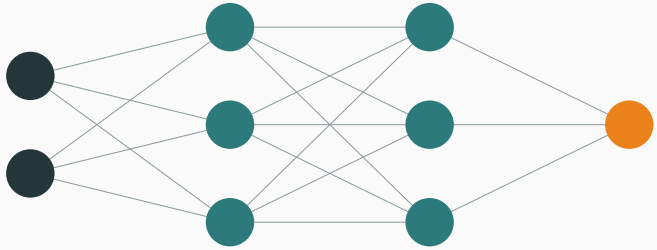
WIDER

One hidden layer, **six neurons**



DEEPER

Two hidden layers, **three neurons each**



Width adds neurons within a layer. Depth adds successive transformations.

Interactive: MLP decision boundaries

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/mlp-decision-boundary

Train a linear classifier or a small MLP on **XOR / circles / spirals / moons** and watch the boundary form. Controls: dataset · linear/MLP · hidden width · one/two layers · activation · train/reset/resample.

Interactive: MLP decision boundaries

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/mlp-decision-boundary

Train a linear classifier or a small MLP on **XOR / circles / spirals / moons** and watch the boundary form. Controls: dataset · linear/MLP · hidden width · one/two layers · activation · train/reset/resample.

Compare wider layers (more features at one stage) with deeper layers (more composed stages).

Why MLPs represent complex functions

A ReLU contributes one hinge

V VISUAL

$$\varphi(z) = \max(0, z)$$

A ReLU contributes one hinge

V VISUAL

$$\varphi(z) = \max(0, z)$$

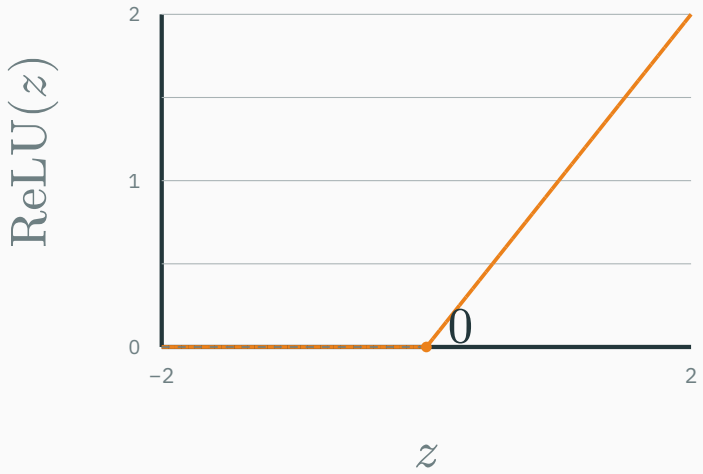
- $z < 0$: output is zero;
- $z > 0$: output grows linearly;
- at $z = 0$: the slope changes—a **hinge**.

A ReLU contributes one hinge

V VISUAL

$$\varphi(z) = \max(0, z)$$

- $z < 0$: output is zero;
- $z > 0$: output grows linearly;
- at $z = 0$: the slope changes—a **hinge**.

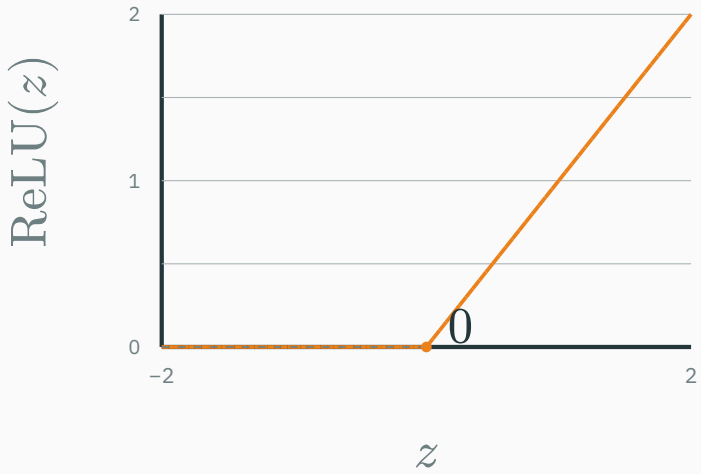


A ReLU contributes one hinge

V VISUAL

$$\varphi(z) = \max(0, z)$$

- $z < 0$: output is zero;
- $z > 0$: output grows linearly;
- at $z = 0$: the slope changes—a **hinge**.



A hidden layer combines many shifted hinges into a piecewise-linear function.

Step 1: one hidden layer creates three shifted ReLUs

V VISUAL

Unit 1 uses weight 1 and bias 0: $u_1 = 1x + 0, \quad h_1 = \mathrm{ReLU}(u_1) = \mathrm{ReLU}(x).$

Step 1: one hidden layer creates three shifted ReLUs

V VISUAL

Unit 1 uses weight 1 and bias 0: $u_1 = 1x + 0, \quad h_1 = \mathrm{ReLU}(u_1) = \mathrm{ReLU}(x).$

Unit 2 uses weight 1 and bias -1: $u_2 = 1x - 1, \quad h_2 = \mathrm{ReLU}(u_2) = \mathrm{ReLU}(x - 1).$

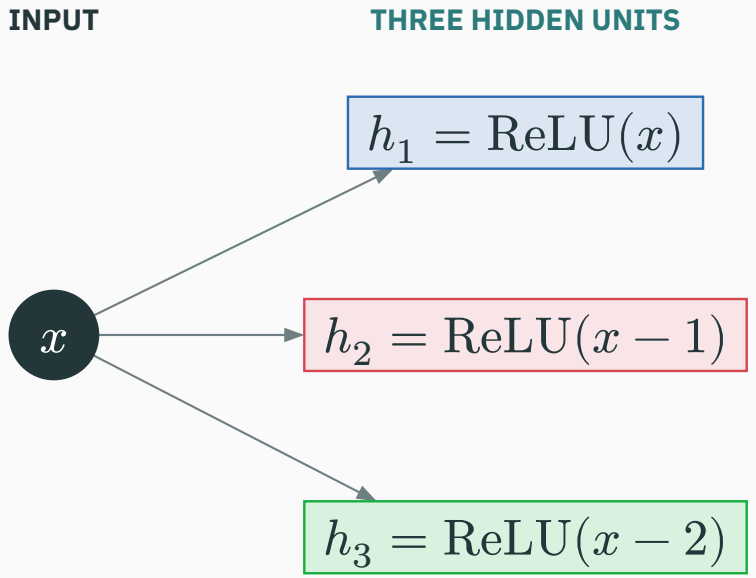
Step 1: one hidden layer creates three shifted ReLUs

V VISUAL

Unit 1 uses weight 1 and bias 0: $u_1 = 1x + 0, \quad h_1 = \mathrm{ReLU}(u_1) = \mathrm{ReLU}(x)$.

Unit 2 uses weight 1 and bias -1: $u_2 = 1x - 1, \quad h_2 = \mathrm{ReLU}(u_2) = \mathrm{ReLU}(x - 1)$.

Unit 3 uses weight 1 and bias -2: $u_3 = 1x - 2, \quad h_3 = \mathrm{ReLU}(u_3) = \mathrm{ReLU}(x - 2)$.



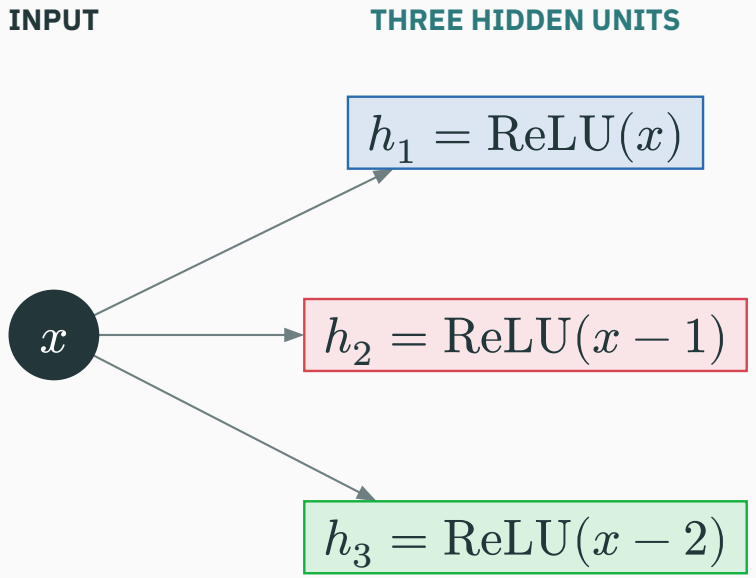
Step 1: one hidden layer creates three shifted ReLUs

V VISUAL

Unit 1 uses weight 1 and bias 0: $u_1 = 1x + 0, \quad h_1 = \mathrm{ReLU}(u_1) = \mathrm{ReLU}(x)$.

Unit 2 uses weight 1 and bias -1: $u_2 = 1x - 1, \quad h_2 = \mathrm{ReLU}(u_2) = \mathrm{ReLU}(x - 1)$.

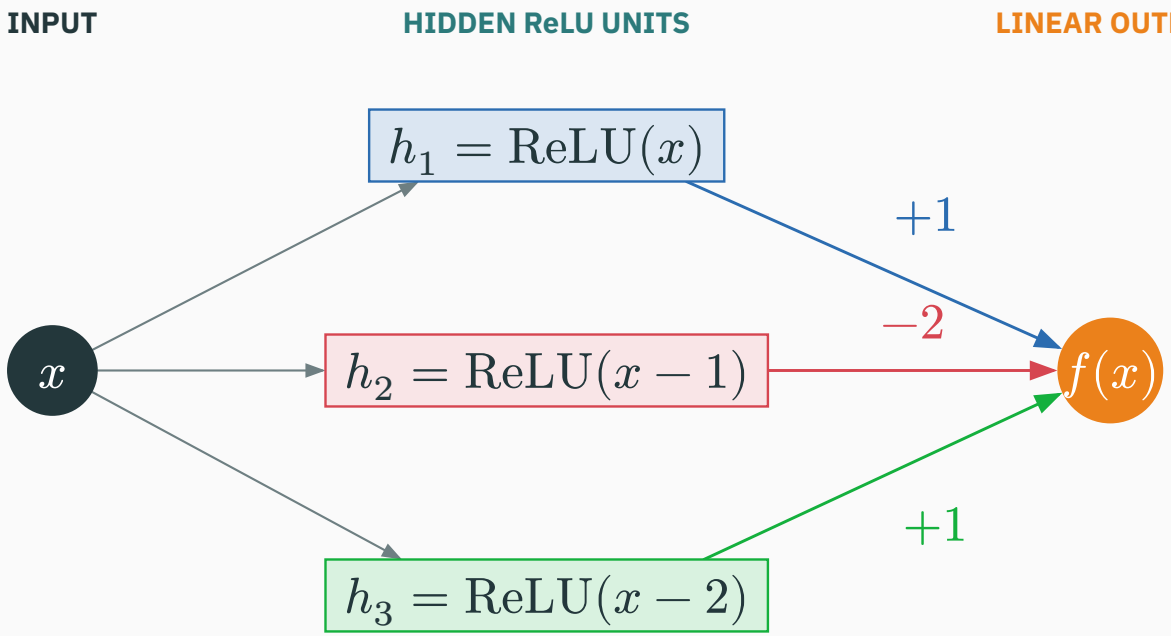
Unit 3 uses weight 1 and bias -2: $u_3 = 1x - 2, \quad h_3 = \mathrm{ReLU}(u_3) = \mathrm{ReLU}(x - 2)$.



The hidden biases move the three hinge locations to $x = 0, 1, 2$. No factor -2 has appeared yet.

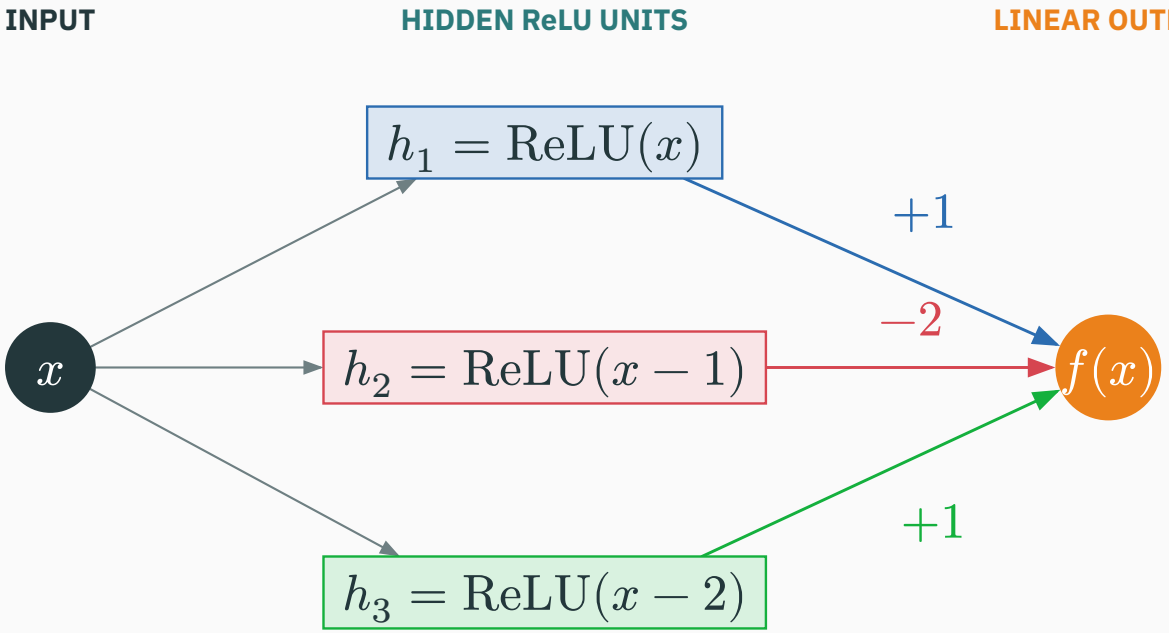
Step 2: the output layer supplies the coefficients

V VISUAL



Step 2: the output layer supplies the coefficients

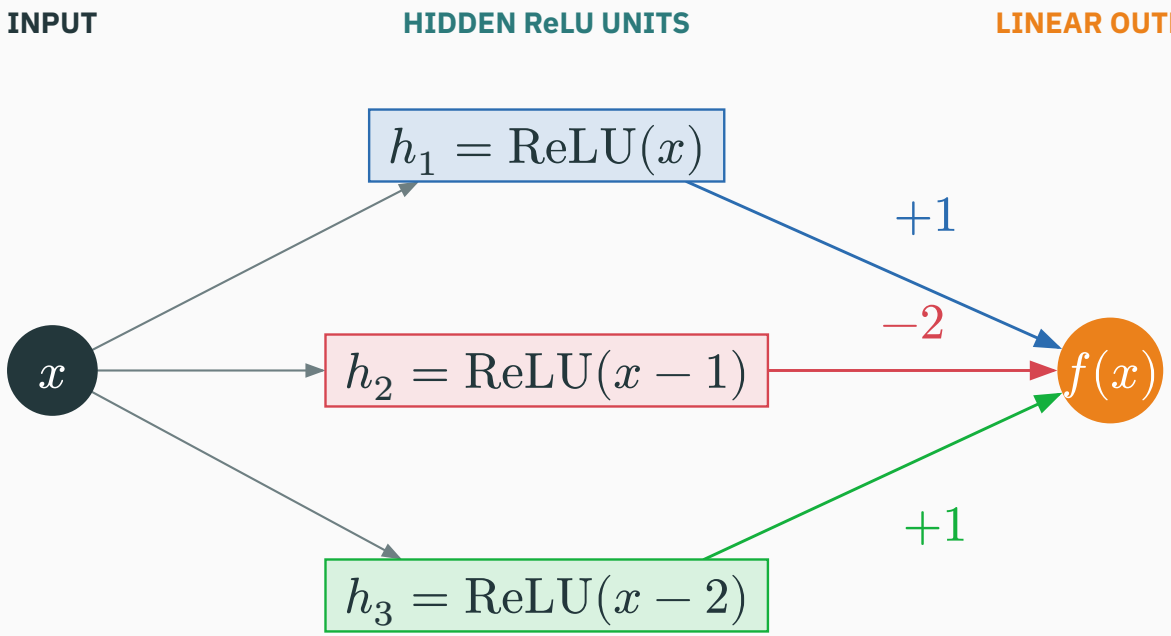
V VISUAL



$$\boldsymbol{h} = \mathrm{ReLU}\left(\begin{pmatrix}1\\1\\1\end{pmatrix}x + \begin{pmatrix}0\\-1\\-2\end{pmatrix}\right), \quad f(x) = \begin{pmatrix}1 & -2 & 1\end{pmatrix}\boldsymbol{h}.$$

Step 2: the output layer supplies the coefficients

V VISUAL



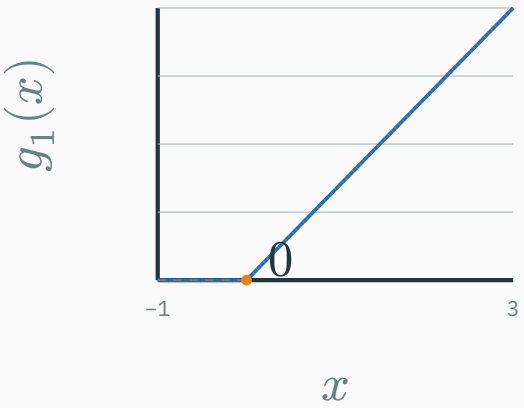
$$\boldsymbol{h} = \mathrm{ReLU}\left(\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} x + \begin{pmatrix} 0 \\ -1 \\ -2 \end{pmatrix} \right), \quad f(x) = \begin{pmatrix} 1 & -2 & 1 \end{pmatrix} \boldsymbol{h}.$$

The -2 is an output-layer weight: the second hidden activation is multiplied by -2 only after ReLU.

Step 3: inspect the output contributions separately

V VISUAL

$$g_1(x) = (+1)h_1$$

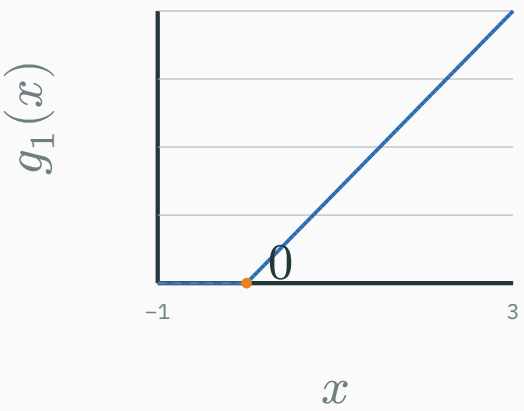


hinge at $x = 0$

Step 3: inspect the output contributions separately

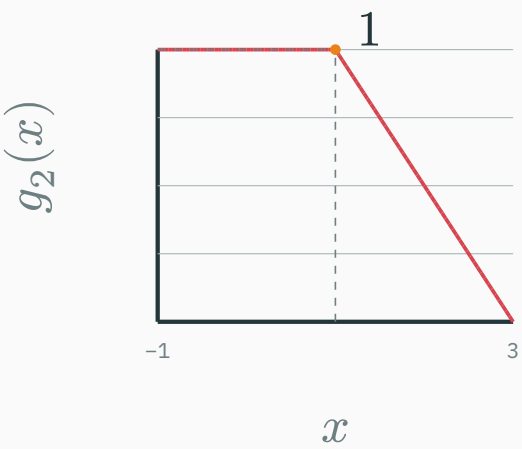
V VISUAL

$g_1(x) = (+1)h_1$



hinge at $x = 0$

$g_2(x) = (-2)h_2$

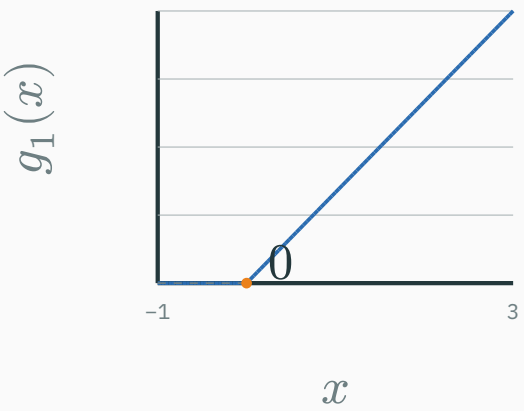


$h_2 \geq 0$; output weight -2 flips it

Step 3: inspect the output contributions separately

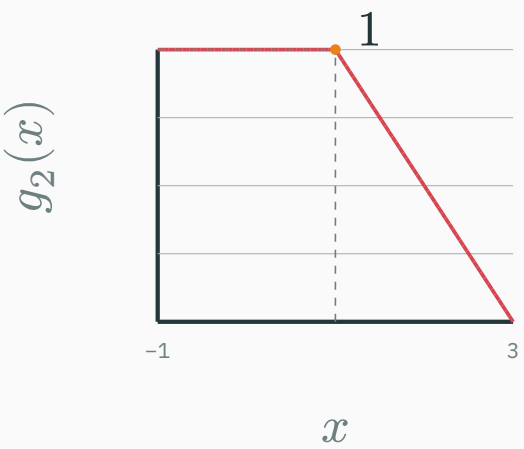
V VISUAL

$g_1(x) = (+1)h_1$



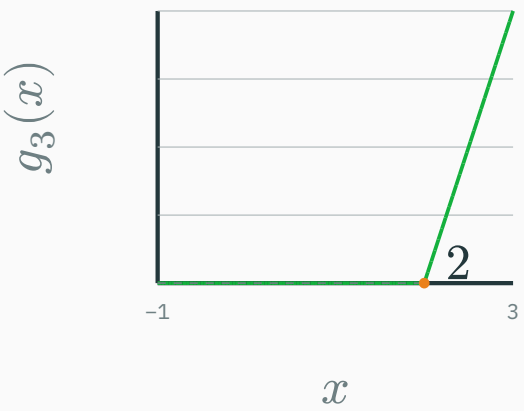
hinge at $x = 0$

$g_2(x) = (-2)h_2$



$h_2 \geq 0$; output weight -2 flips it

$g_3(x) = (+1)h_3$

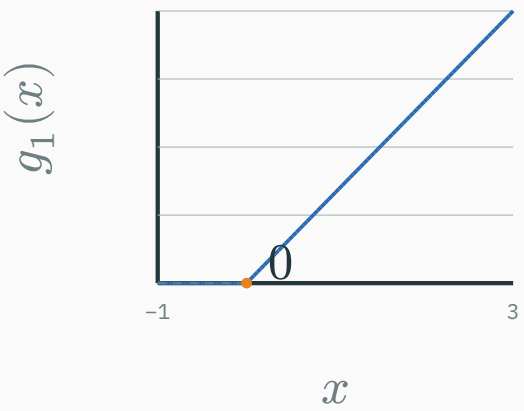


hinge at $x = 2$

Step 3: inspect the output contributions separately

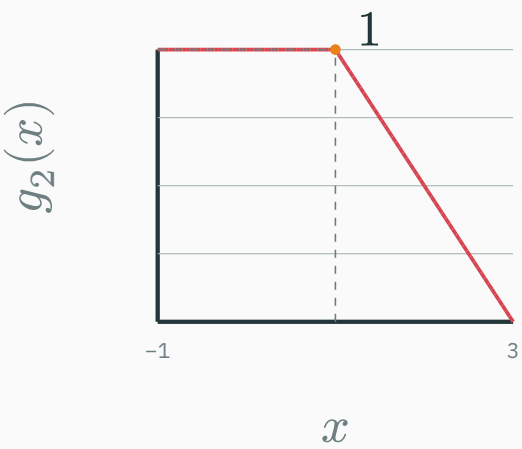
V VISUAL

$g_1(x) = (+1)h_1$



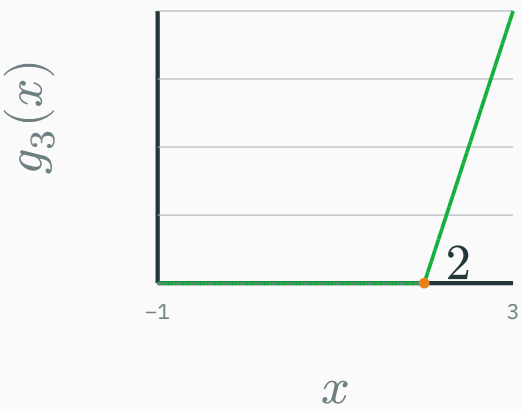
hinge at $x = 0$

$g_2(x) = (-2)h_2$



$h_2 \geq 0$; output weight -2 flips it

$g_3(x) = (+1)h_3$



hinge at $x = 2$

Hidden activations h_j are the features; $g_j = v_j h_j$ are their signed contributions to the output.

Step 4: add the active terms on each interval

D DERIVATION

$$f(x) = h_1 - 2h_2 + h_3 = \mathrm{ReLU}(x) - 2\mathrm{ReLU}(x-1) + \mathrm{ReLU}(x-2).$$

Step 4: add the active terms on each interval

D DERIVATION

$$f(x) = h_1 - 2h_2 + h_3 = \mathrm{ReLU}(x) - 2\mathrm{ReLU}(x-1) + \mathrm{ReLU}(x-2).$$

Input region	h_1	$-2h_2$	h_3	Sum $f(x)$
$x < 0$	0	0	0	0
$0 \le x < 1$	x	0	0	x
$1 \le x < 2$	x	$-2(x-1)$	0	$2 - x$
$x \ge 2$	x	$-2(x-1)$	$x - 2$	0

Step 4: add the active terms on each interval

D DERIVATION

$$f(x) = h_1 - 2h_2 + h_3 = \mathrm{ReLU}(x) - 2\mathrm{ReLU}(x-1) + \mathrm{ReLU}(x-2).$$

Input region	h_1	$-2h_2$	h_3	Sum $f(x)$
$x < 0$	0	0	0	0
$0 \le x < 1$	x	0	0	x
$1 \le x < 2$	x	$-2(x-1)$	0	$2 - x$
$x \ge 2$	x	$-2(x-1)$	$x - 2$	0

The output rises as x, falls as $2 - x$, then returns to zero: a tent.

Step 5: the weighted sum is the tent

V VISUAL

Hidden layer $\boldsymbol{h}(x) =$
$(\mathrm{ReLU}(x), \mathrm{ReLU}(x-1), \mathrm{ReLU}(x-2)).$

Step 5: the weighted sum is the tent

V VISUAL

Hidden layer $\boldsymbol{h}(x) =$
$(\mathrm{ReLU}(x), \mathrm{ReLU}(x-1), \mathrm{ReLU}(x-2))$.

Linear output layer $f(x) =$
$\begin{pmatrix} 1 & -2 & 1 \end{pmatrix} \boldsymbol{h}(x)$.

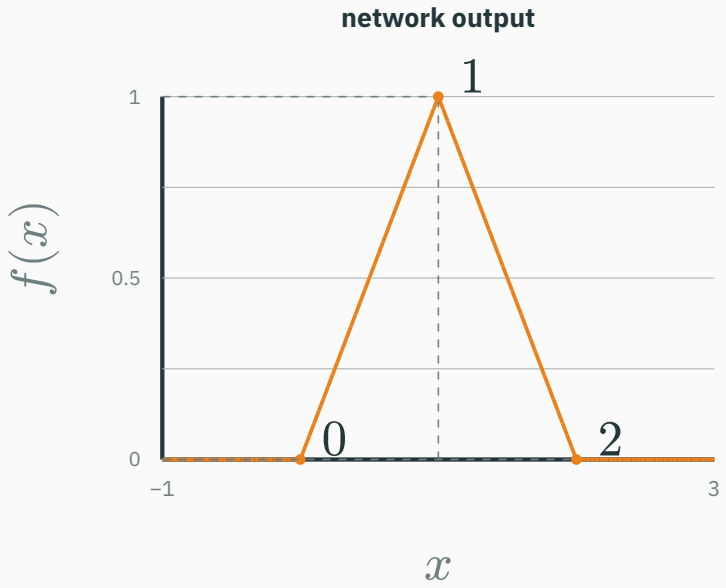
Step 5: the weighted sum is the tent

V VISUAL

Hidden layer $\boldsymbol{h}(x) = (\mathrm{ReLU}(x), \mathrm{ReLU}(x-1), \mathrm{ReLU}(x-2))$.

Linear output layer $f(x) = \begin{pmatrix} 1 & -2 & 1 \end{pmatrix} \boldsymbol{h}(x)$.

The network has **three hidden nodes** and **one output node**. The output node performs no ReLU here; it only takes a weighted sum.



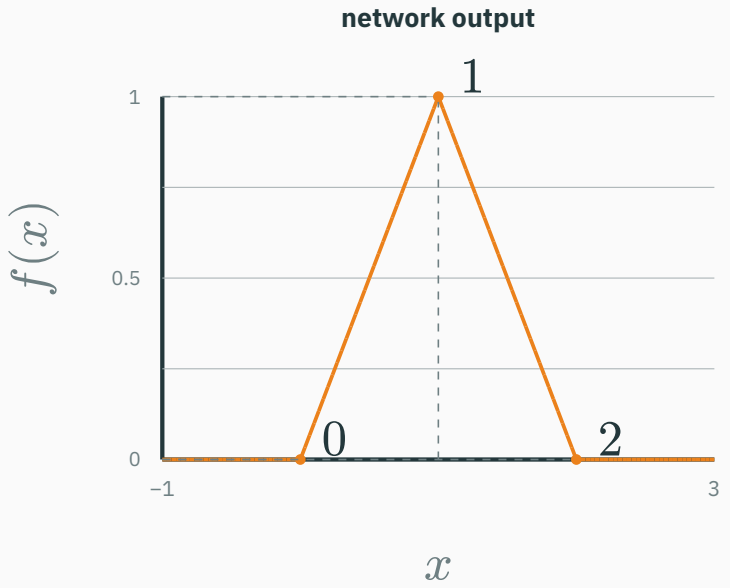
Step 5: the weighted sum is the tent

V VISUAL

Hidden layer $\boldsymbol{h}(x) = (\mathrm{ReLU}(x), \mathrm{ReLU}(x-1), \mathrm{ReLU}(x-2))$.

Linear output layer $f(x) = \begin{pmatrix} 1 & -2 & 1 \end{pmatrix} \boldsymbol{h}(x)$.

The network has **three hidden nodes** and **one output node**. The output node performs no ReLU here; it only takes a weighted sum.



Hidden units create reusable nonlinear features; the output layer chooses their signs and strengths.

Five hinges create a two-peak function

V VISUAL

A slightly richer weighted sum:

$$q(x) = \mathrm{ReLU}(x) - 2\mathrm{ReLU}(x-1)$$
$$+2\mathrm{ReLU}(x-2) - 2\mathrm{ReLU}(x-3)$$
$$+ \mathrm{ReLU}(x-4).$$

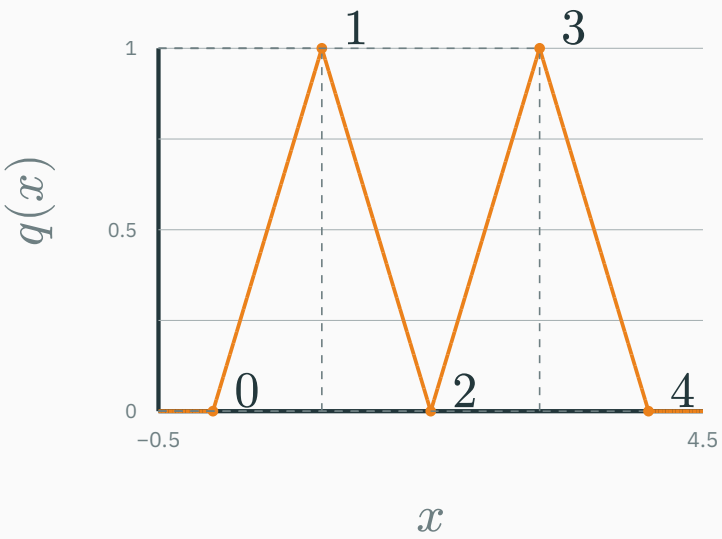
Five hinges create a two-peak function

V VISUAL

A slightly richer weighted sum:

$$q(x) = \mathrm{ReLU}(x) - 2\mathrm{ReLU}(x-1)$$
$$+2\mathrm{ReLU}(x-2) - 2\mathrm{ReLU}(x-3)$$
$$+ \mathrm{ReLU}(x-4).$$

Each shifted unit changes the slope at one breakpoint.



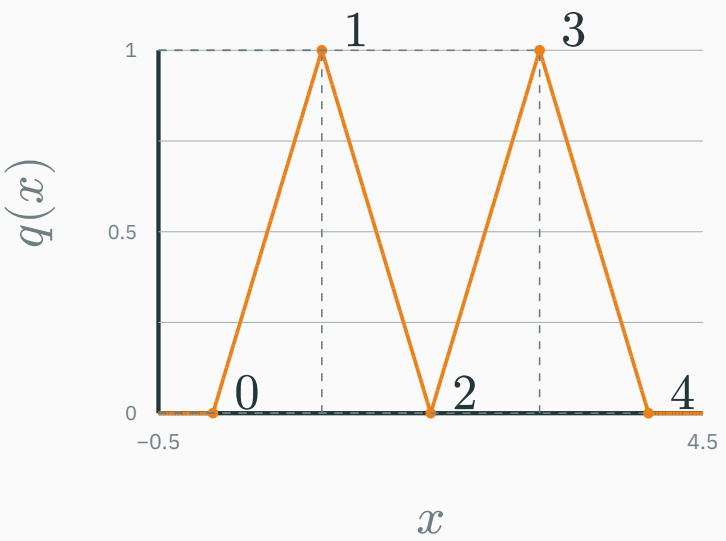
Five hinges create a two-peak function

V VISUAL

A slightly richer weighted sum:

$$q(x) = \mathrm{ReLU}(x) - 2\mathrm{ReLU}(x-1)$$
$$+2\mathrm{ReLU}(x-2) - 2\mathrm{ReLU}(x-3)$$
$$+ \mathrm{ReLU}(x-4).$$

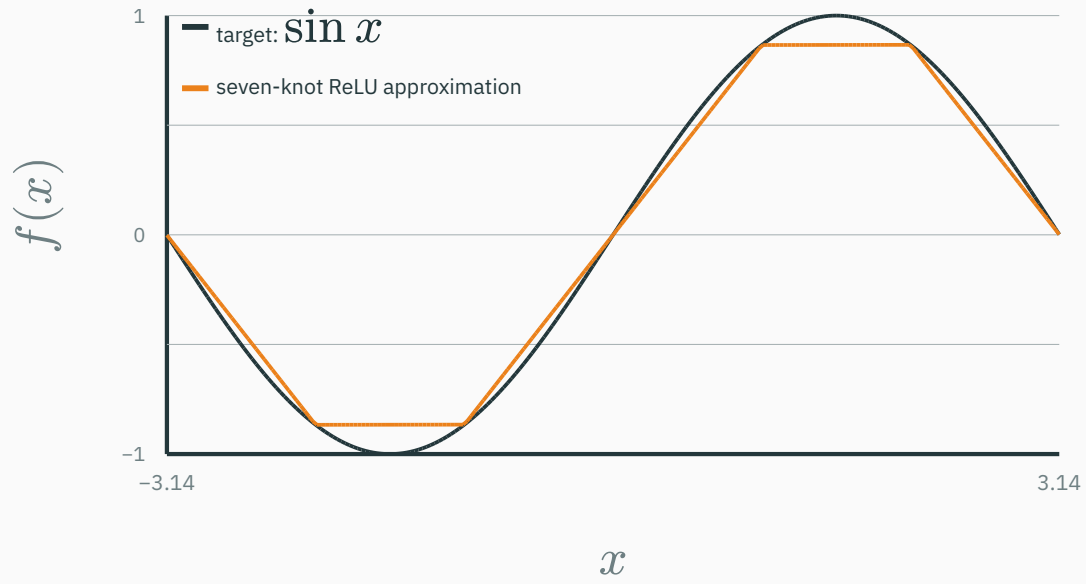
Each shifted unit changes the slope at one breakpoint.



More hidden units → more breakpoints → a more detailed piecewise-linear function.

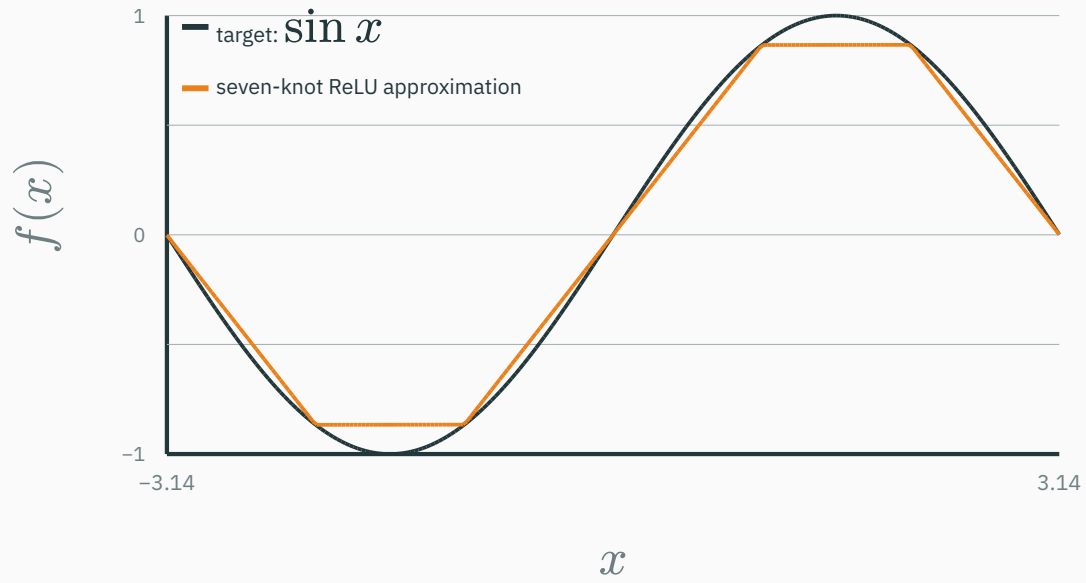
More breakpoints can follow a smooth curve

V VISUAL



More breakpoints can follow a smooth curve

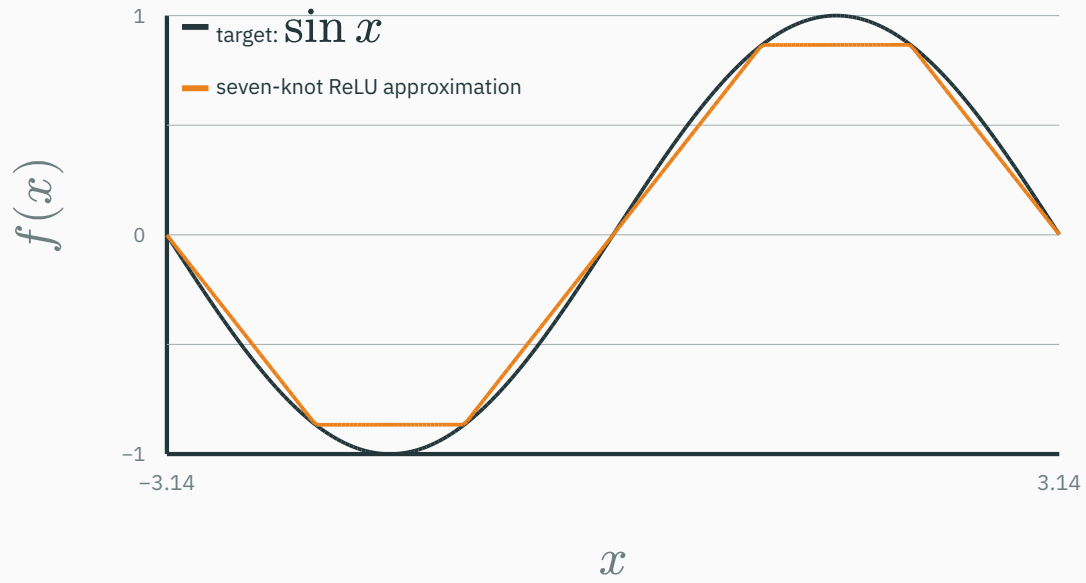
V VISUAL



The orange approximation is still made only of straight segments; its corners are the hinges.

More breakpoints can follow a smooth curve

V VISUAL



The orange approximation is still made only of straight segments; its corners are the hinges.

Adding hidden units adds breakpoints and reduces the approximation gap.

Interactive: build functions from ReLU hinges

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/dl-teaching/interactives/relu-function-builder.html

Recreate the sequence: inspect one hinge, combine three into a tent, add two more for a two-peak function, then increase the number of knots used to approximate $\sin x$. Controls: preset · hidden-unit count · hinge location · output weight · show components.

Interactive: build functions from ReLU hinges

I INTERACTIVE

INTERACTIVE

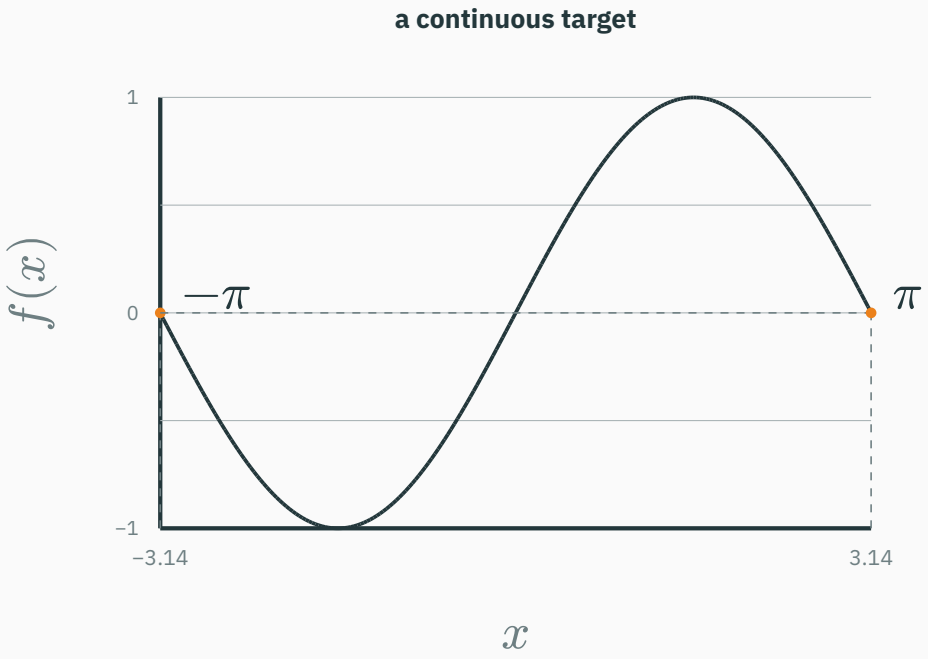
↗ nipunbatra.github.io/dl-teaching/interactives/relu-function-builder.html

Recreate the sequence: inspect one hinge, combine three into a tent, add two more for a two-peak function, then increase the number of knots used to approximate $\sin x$. Controls: preset · hidden-unit count · hinge location · output weight · show components.

Predict which local slope will change **before** moving a hinge or its output weight.

First fix the target and the input region

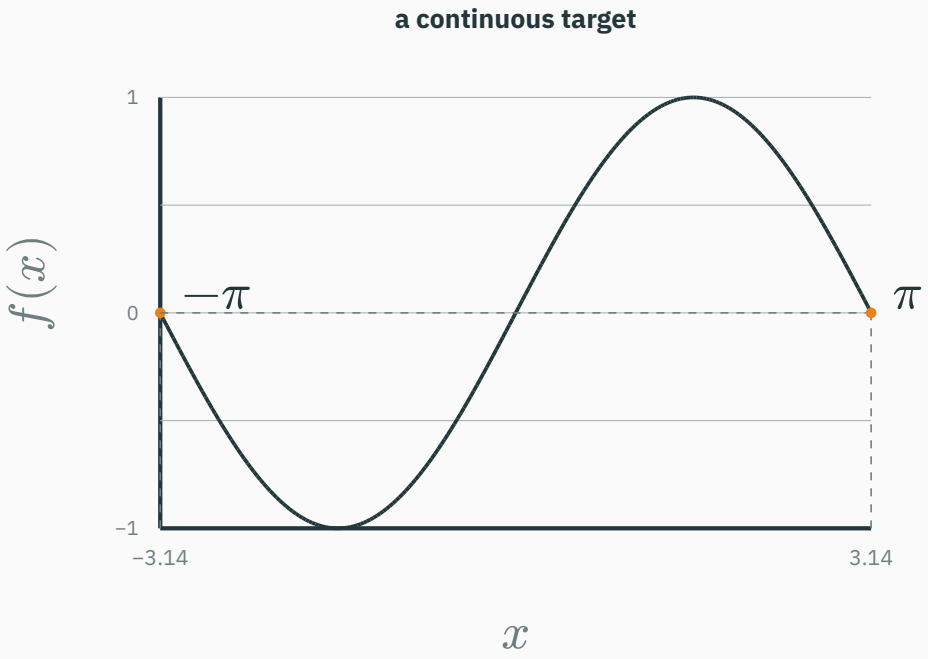
V VISUAL



Continuous means that the target has no jumps.

First fix the target and the input region

V VISUAL



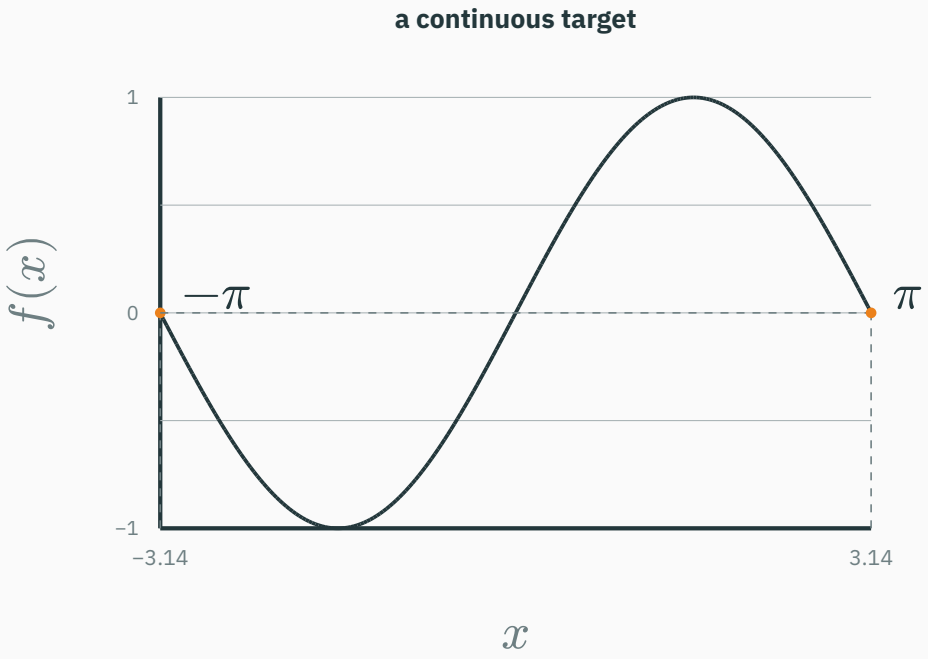
Continuous means that the target has no jumps.

In one dimension, a **compact domain** can be a closed, bounded interval such as

$$D = [-\pi, \pi].$$

First fix the target and the input region

V VISUAL



Continuous means that the target has no jumps.

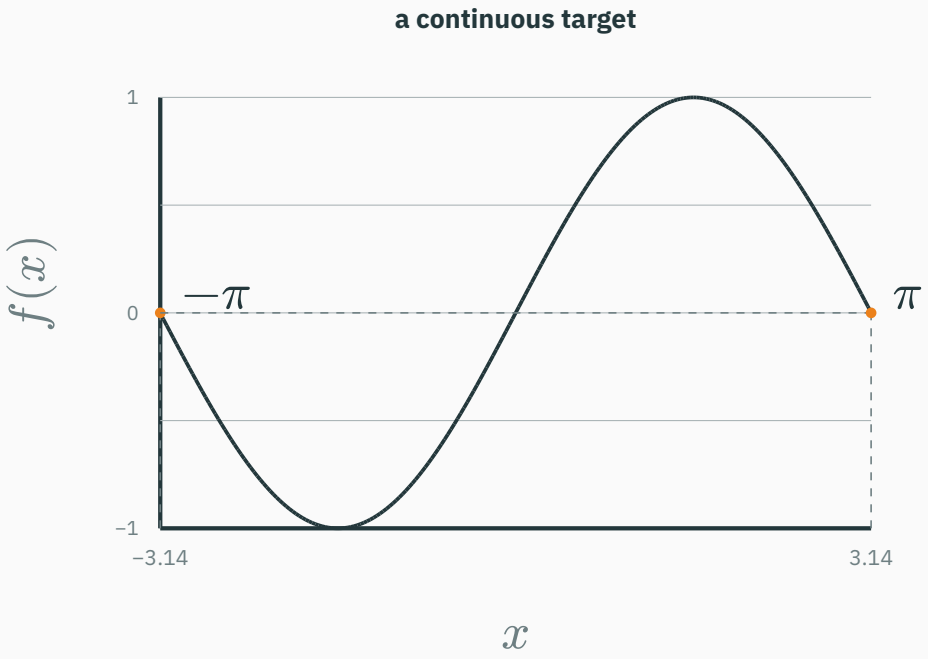
In one dimension, a **compact domain** can be a closed, bounded interval such as

$$D = [-\pi, \pi].$$

We ask the network to match f **on** D; the theorem makes no claim outside that chosen region.

First fix the target and the input region

V VISUAL



Continuous means that the target has no jumps.

In one dimension, a **compact domain** can be a closed, bounded interval such as

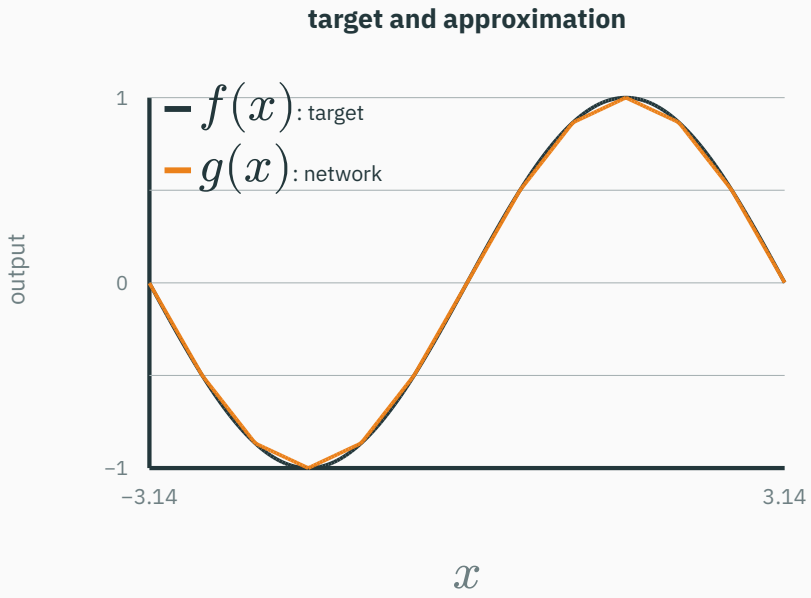
$$D = [-\pi, \pi].$$

We ask the network to match f **on** D; the theorem makes no claim outside that chosen region.

Approximation begins with one target function on one specified input region.

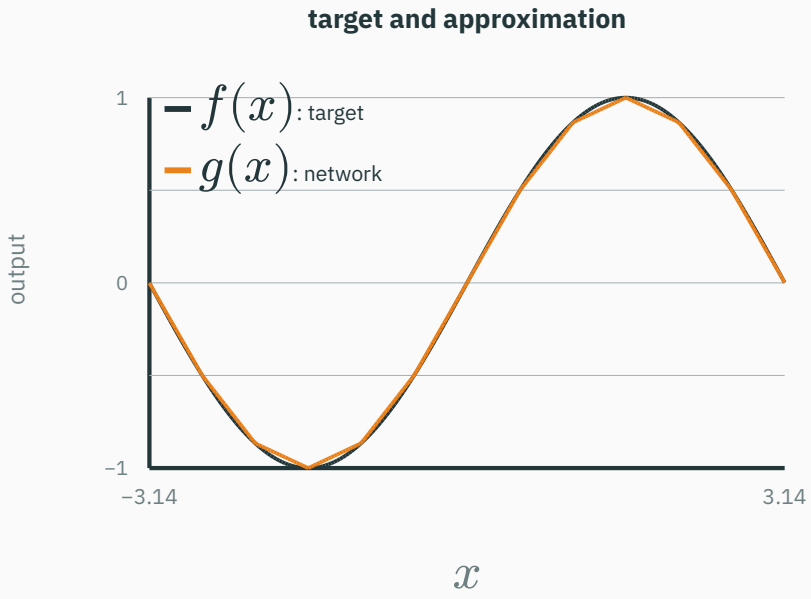
“Within ε” means close at every input

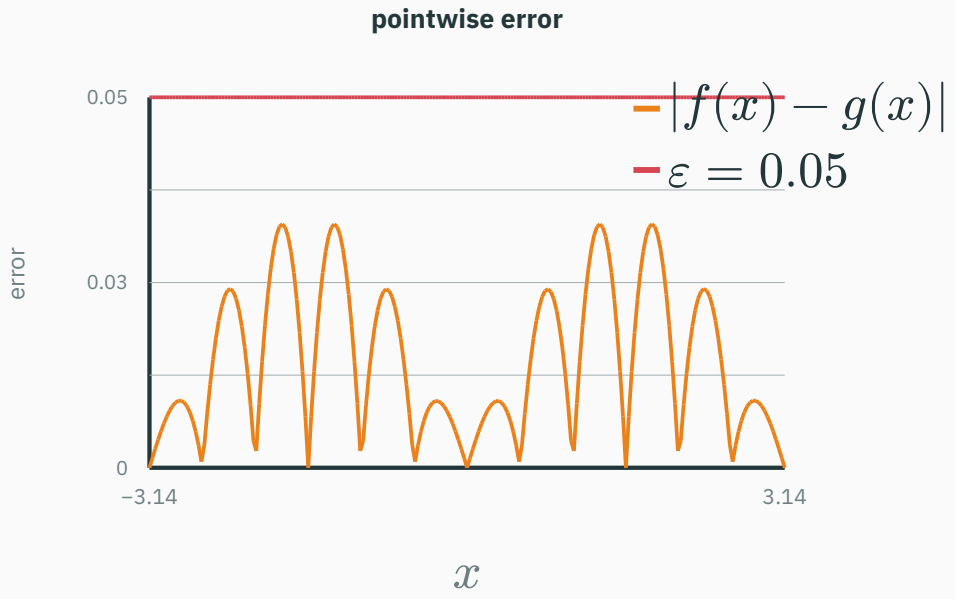
V VISUAL



“Within ε” means close at every input

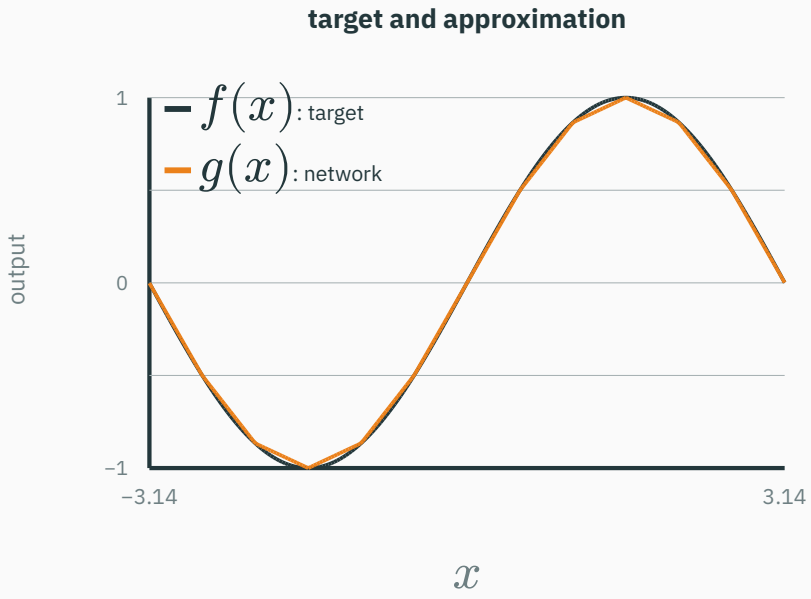
V VISUAL

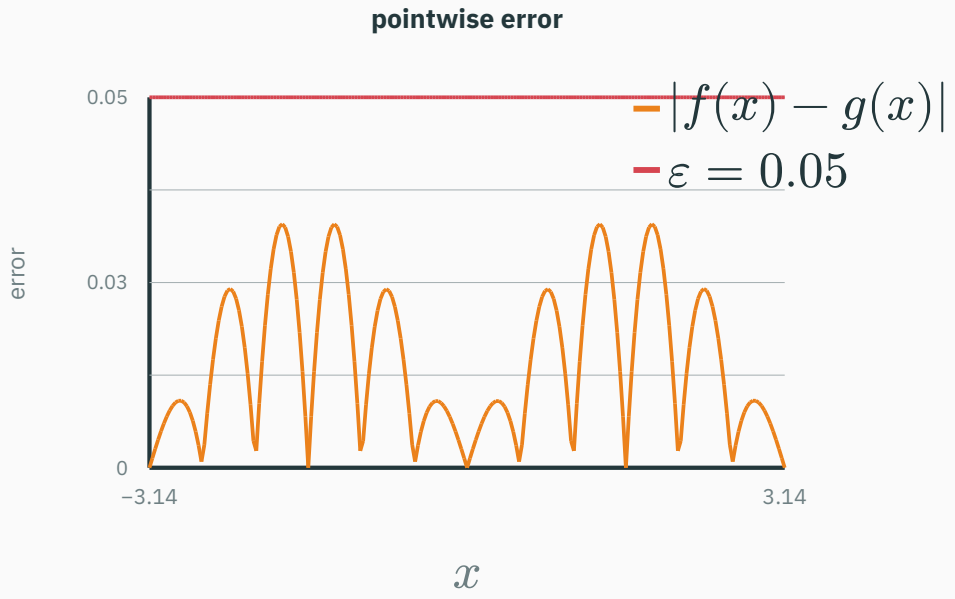




“Within ε” means close at every input

V VISUAL

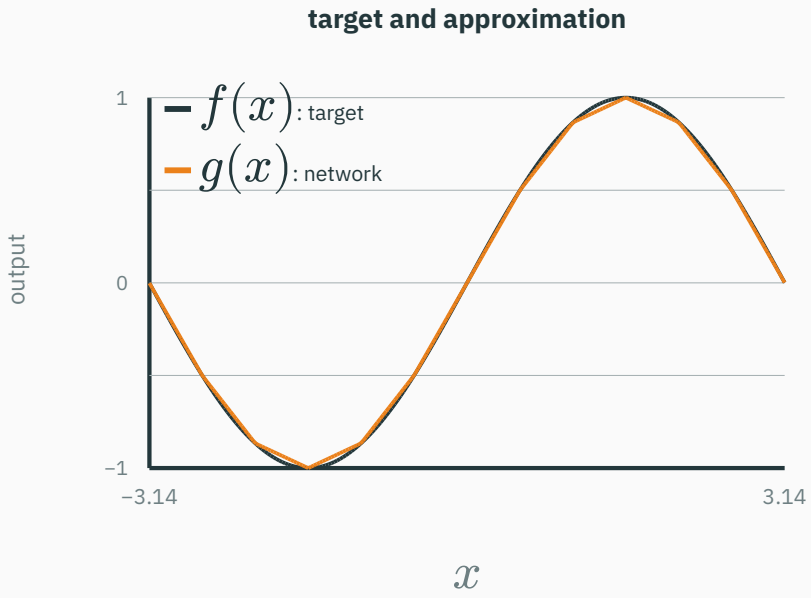


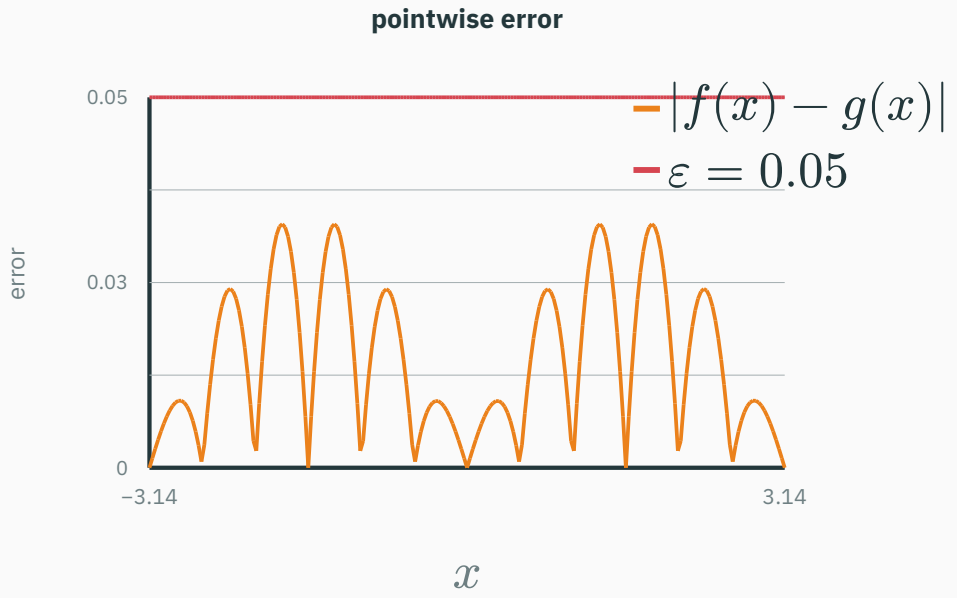


$\sup_{x \in D} |f(x) - g(x)|$ is the **largest vertical gap** anywhere on D.

“Within ε” means close at every input

V VISUAL





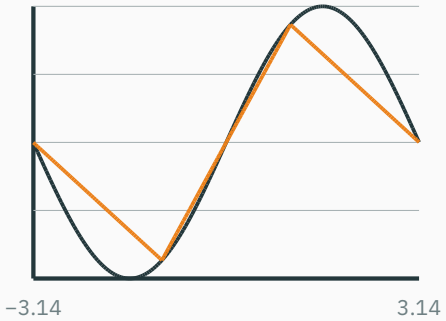
$\sup_{x \in D} |f(x) - g(x)|$ is the **largest vertical gap** anywhere on D.

The guarantee is uniform: no input in the interval may exceed the chosen tolerance.

More hinges reduce the largest approximation error

V VISUAL

4 knots



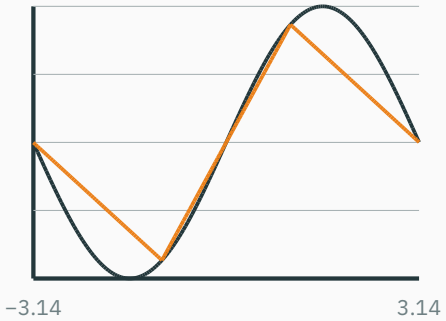
x

largest error ≈ 0.44

More hinges reduce the largest approximation error

V VISUAL

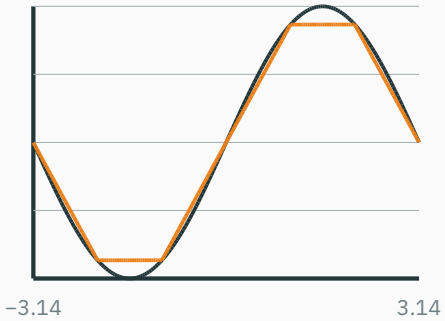
4 knots



x

largest error ≈ 0.44

7 knots



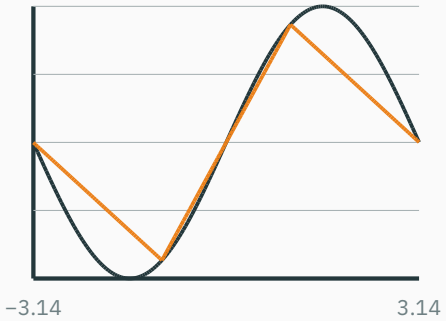
x

largest error ≈ 0.13

More hinges reduce the largest approximation error

V VISUAL

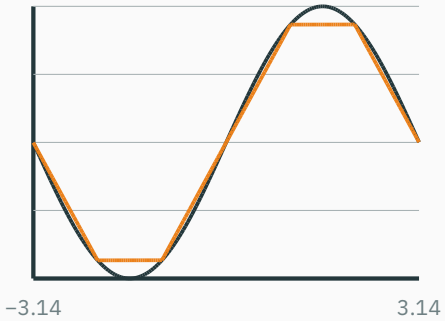
4 knots



x

largest error ≈ 0.44

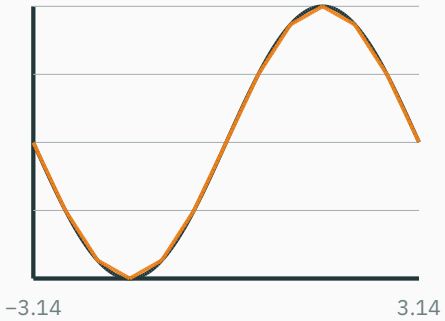
7 knots



x

largest error ≈ 0.13

13 knots



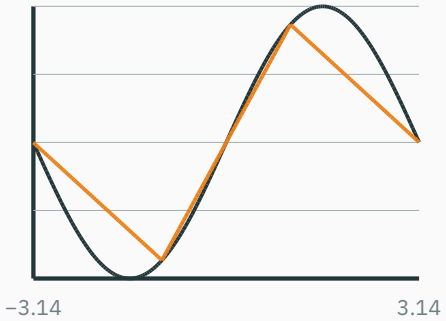
x

largest error ≈ 0.033

More hinges reduce the largest approximation error

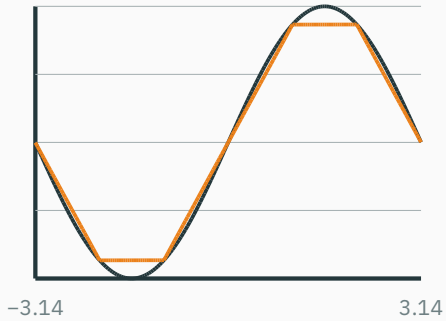
V VISUAL

4 knots



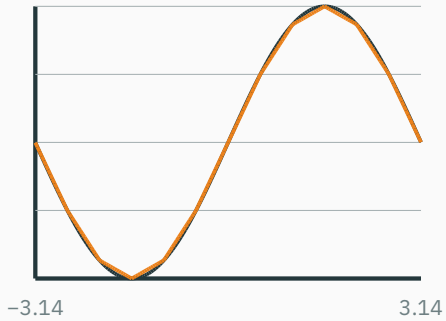
largest error ≈ 0.44

7 knots



largest error ≈ 0.13

13 knots



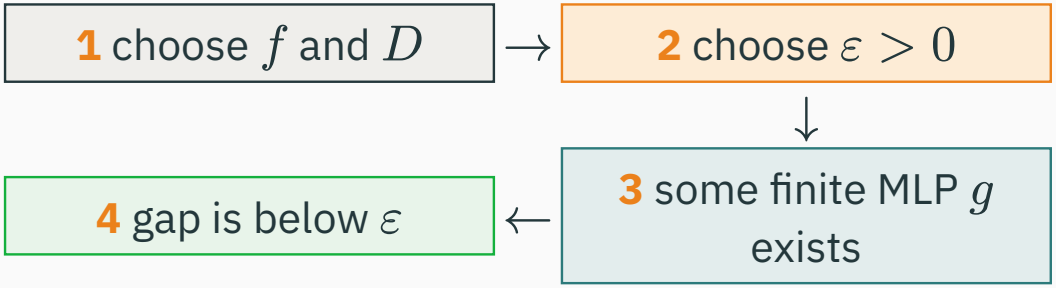
largest error ≈ 0.033

More hidden units provide more breakpoints, so a piecewise-linear network can follow finer detail.

Read the universal-approximation claim in order

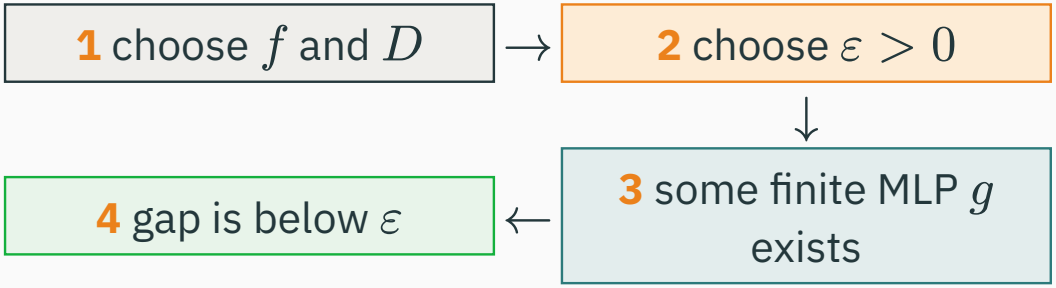
D DERIVATION

The order matters.



Read the universal-approximation claim in order

D DERIVATION

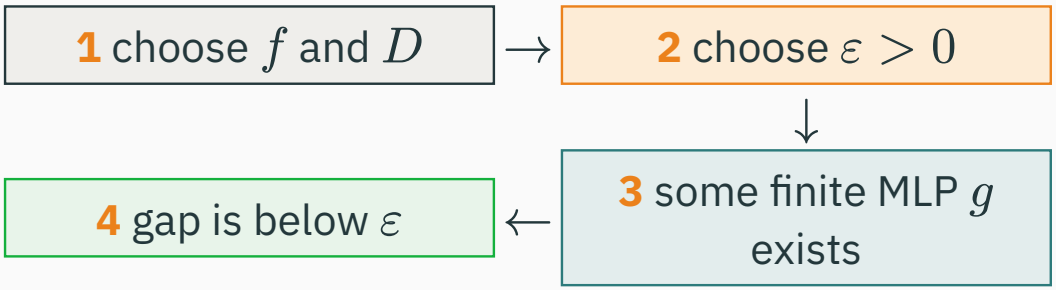


The order matters.

For every ε: we may demand any positive tolerance.

Read the universal-approximation claim in order

D DERIVATION



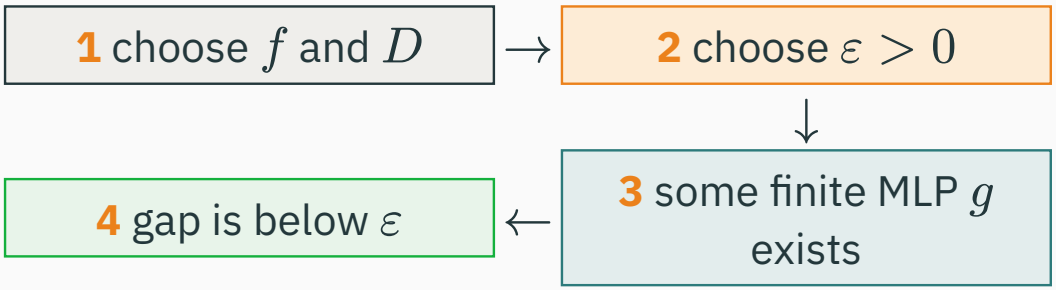
The order matters.

For every ε: we may demand any positive tolerance.

There exists g: the network can change when we demand a smaller error.

Read the universal-approximation claim in order

D DERIVATION



The order matters.

For every ε: we may demand any positive tolerance.

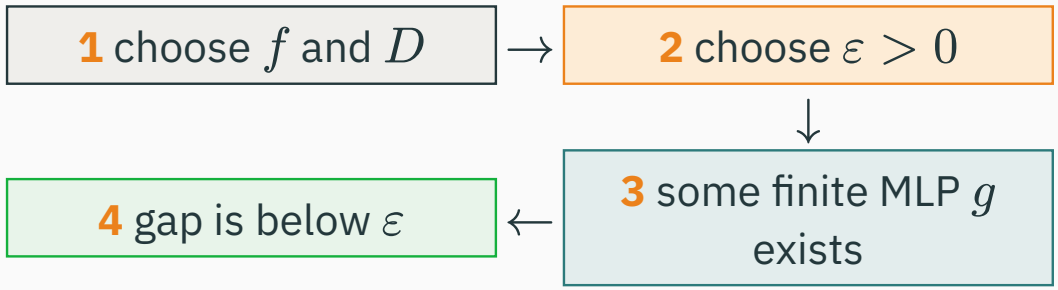
There exists g: the network can change when we demand a smaller error.

$$\forall \varepsilon > 0, \exists\, \mathrm{MLP}\ g$$

$$\text{such that} \quad \sup_{x \in D} |f(x) - g(x)| < \varepsilon.$$

Read the universal-approximation claim in order

D DERIVATION



The order matters.

For every ε: we may demand any positive tolerance.

There exists g: the network can change when we demand a smaller error.

$$\forall \varepsilon > 0, \exists \,\mathrm{MLP}\ g$$

$$\text{such that} \quad \sup_{x \in D} |f(x) - g(x)| < \varepsilon.$$

This is a statement about representational capacity—not yet about how to find the network.

Existence does not guarantee learnability

★ OPTIONAL

Universal approximation does **not** promise:

Existence does not guarantee learnability

★ OPTIONAL

Universal approximation does **not** promise:

- that training will **find** that function;
- that **finite data** is enough;

Existence does not guarantee learnability

★ OPTIONAL

Universal approximation does **not** promise:

- that training will **find** that function;
- that **finite data** is enough;
- that a **small** network suffices;
- that it will **generalize**.

Existence does not guarantee learnability

★ OPTIONAL

Universal approximation does **not** promise:

- that training will **find** that function;
- that **finite data** is enough;
- that a **small** network suffices;
- that it will **generalize**.

expressivity $\neq$ trainability $\neq$ generalization

Composition means feeding one result into the next

V VISUAL

Start with two small functions:

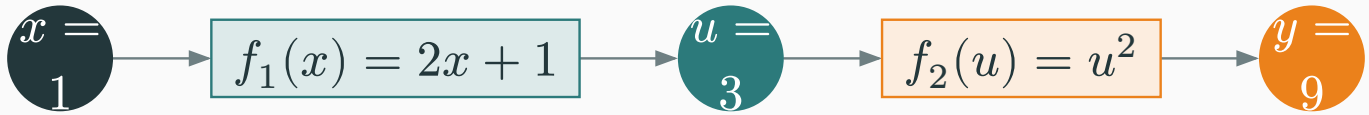
$$f_1(x) = 2x + 1, \quad f_2(u) = u^2.$$

Composition means feeding one result into the next

V VISUAL

Start with two small functions:

$$f_1(x) = 2x + 1, \quad f_2(u) = u^2.$$

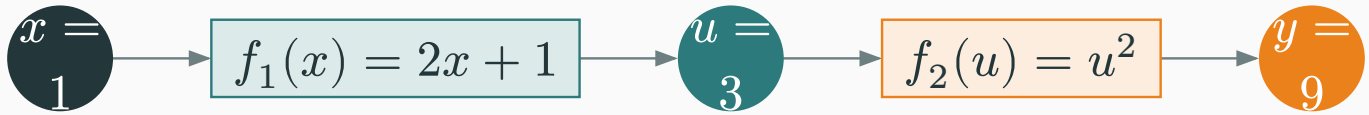


Composition means feeding one result into the next

V VISUAL

Start with two small functions:

$$f_1(x) = 2x + 1, \quad f_2(u) = u^2.$$



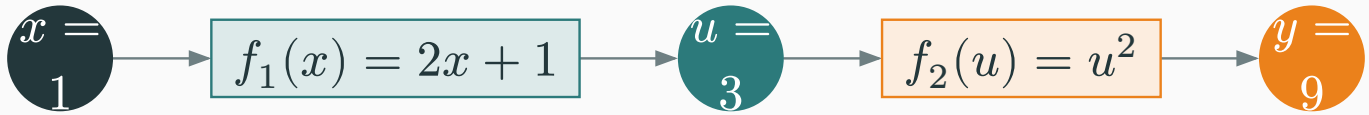
$$y = f_2(f_1(x)) = (2x + 1)^2$$

Composition means feeding one result into the next

V VISUAL

Start with two small functions:

$$f_1(x) = 2x + 1, \quad f_2(u) = u^2.$$



$$y = f_2(f_1(x)) = (2x + 1)^2$$

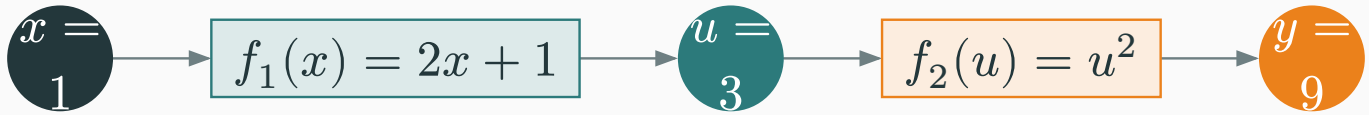
The value u is an **intermediate feature**: the second stage uses what the first stage computed.

Composition means feeding one result into the next

V VISUAL

Start with two small functions:

$$f_1(x) = 2x + 1, \quad f_2(u) = u^2.$$



$$y = f_2(f_1(x)) = (2x + 1)^2$$

The value u is an **intermediate feature**: the second stage uses what the first stage computed.

Composition is sequential reuse: $f = f_2 \circ f_1$.

A deep model can build features in stages

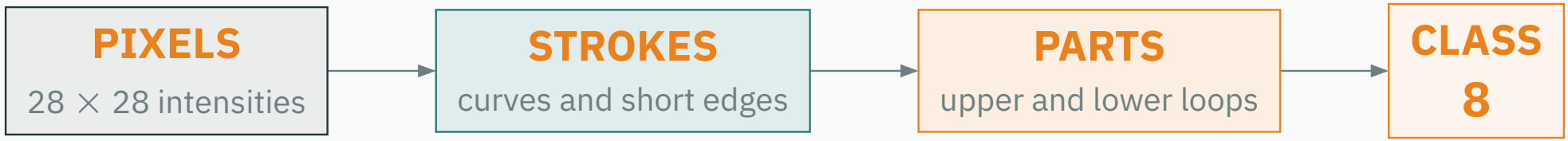
V VISUAL

Consider recognizing a handwritten **8** from an image:

A deep model can build features in stages

V VISUAL

Consider recognizing a handwritten **8** from an image:



A deep model can build features in stages

V VISUAL

Consider recognizing a handwritten **8** from an image:

Layer 1 detects local contrast.

Layer 2 combines strokes into loops.

Output combines parts into a class score.

A deep model can build features in stages

V VISUAL

Consider recognizing a handwritten **8** from an image:

Layer 1 detects local contrast.

Layer 2 combines strokes into loops.

Output combines parts into a class score.

Later features are functions of earlier features: depth mirrors a hierarchy.

Depth can match a compositional problem

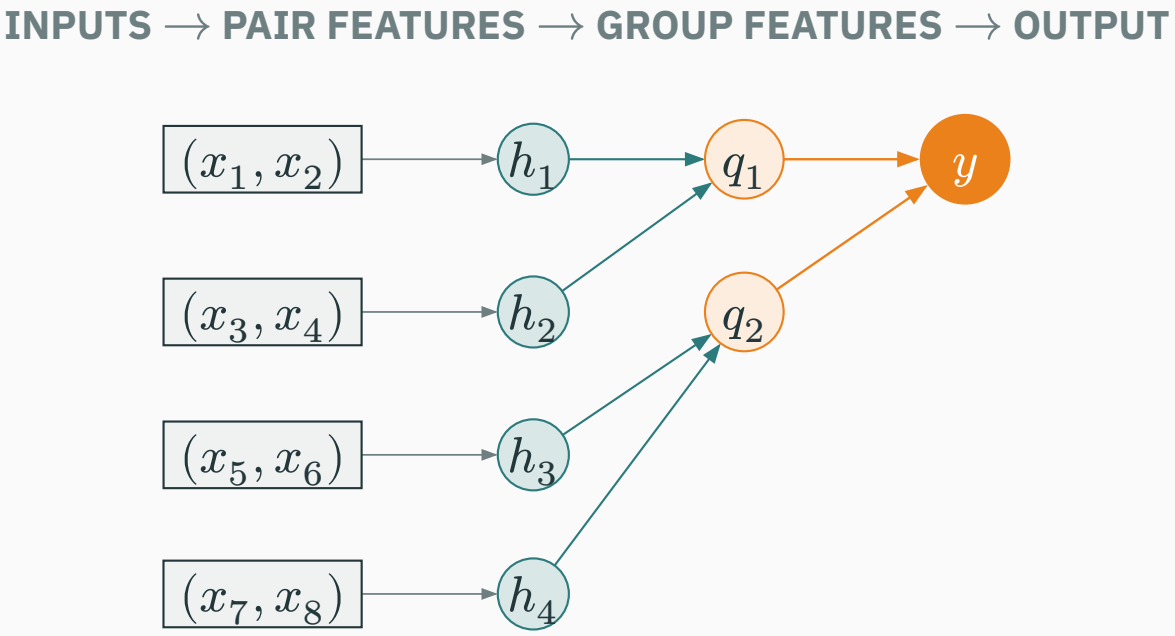
V VISUAL

Suppose the target itself combines eight inputs in stages:

Depth can match a compositional problem

V VISUAL

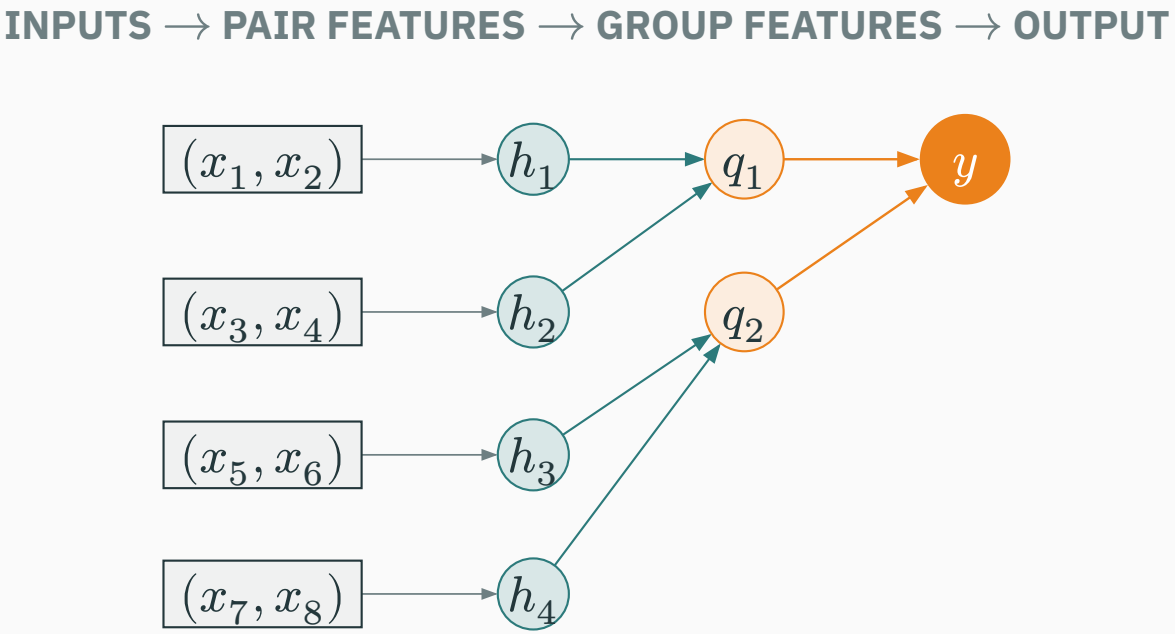
Suppose the target itself combines eight inputs in stages:



Depth can match a compositional problem

V VISUAL

Suppose the target itself combines eight inputs in stages:



Each node combines two values, yet y depends on all eight. For some compositional functions, a shallow network needs far more units.

Connecting to deep-learning practice

A forward pass transforms one input, step by step

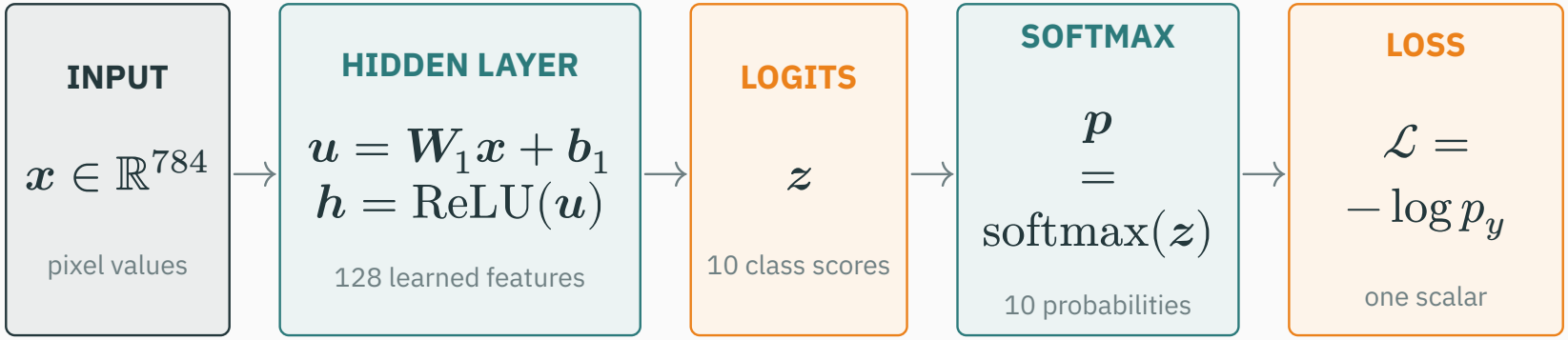
V VISUAL

Example: classify one 28×28 image — 784 pixel values.

A forward pass transforms one input, step by step

V VISUAL

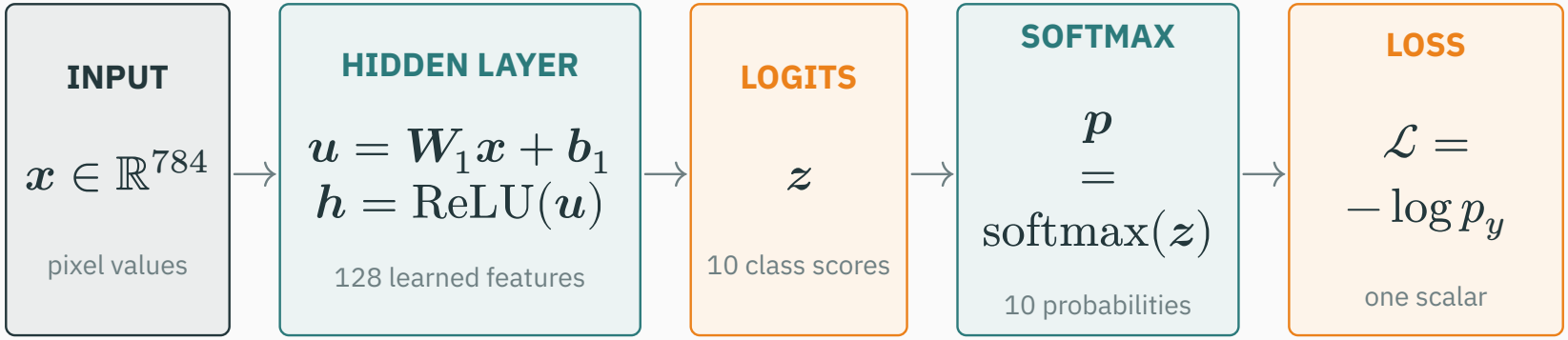
Example: classify one 28×28 image — 784 pixel values.



A forward pass transforms one input, step by step

V VISUAL

Example: classify one 28×28 image — 784 pixel values.

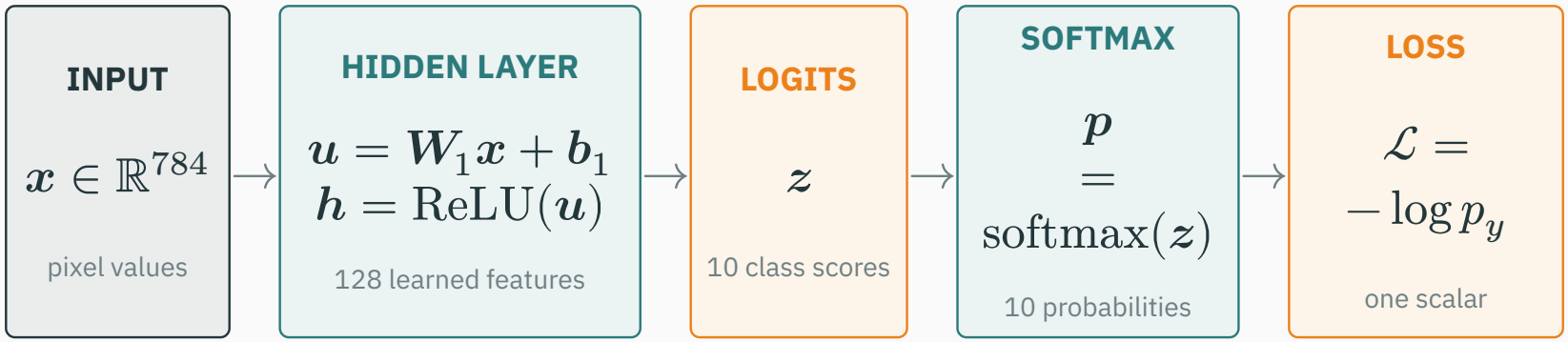


The true class y selects its predicted probability p_y; a smaller p_y gives a larger loss.

A forward pass transforms one input, step by step

V VISUAL

Example: classify one 28×28 image — 784 pixel values.



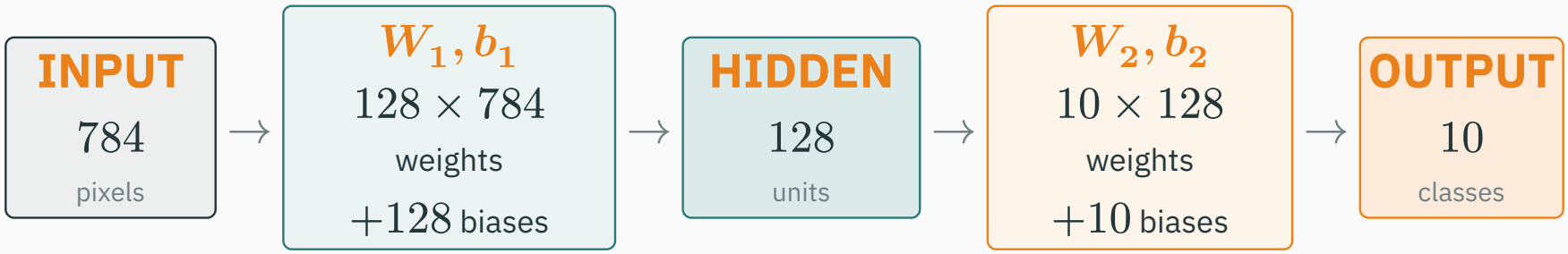
The true class y selects its predicted probability p_y; a smaller p_y gives a larger loss.

A forward pass computes a prediction and its loss; it does not update the weights.

Count parameters by following the same network

D DERIVATION

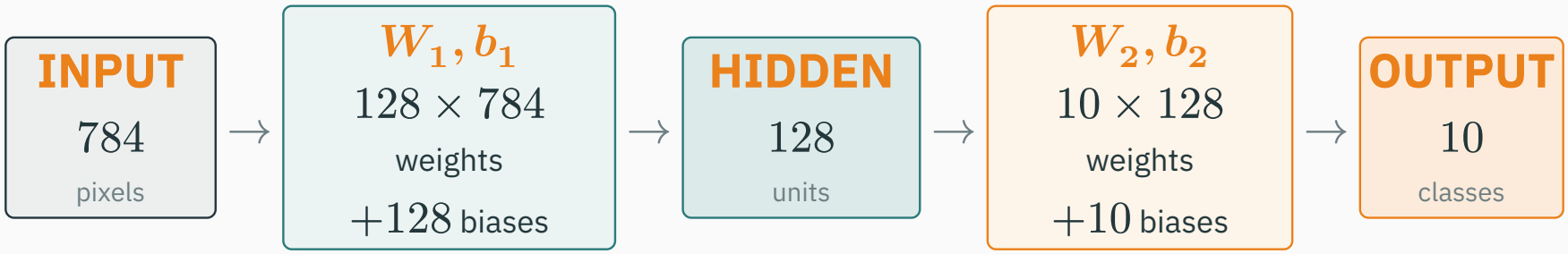
Every connection has a **weight**; every receiving unit has a **bias**.



Count parameters by following the same network

D DERIVATION

Every connection has a **weight**; every receiving unit has a **bias**.



Input → hidden

weights: $128 \times 784 = 100{,}352$

biases: 128

subtotal: $100{,}480$

Hidden → output

weights: $10 \times 128 = 1{,}280$

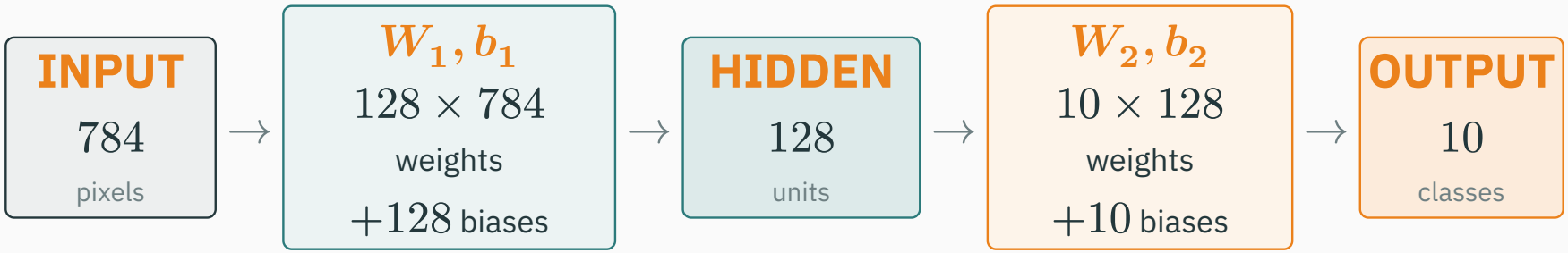
biases: 10

subtotal: $1{,}290$

Count parameters by following the same network

D DERIVATION

Every connection has a **weight**; every receiving unit has a **bias**.



Input → hidden

weights: $128 \times 784 = 100{,}352$

biases: 128

subtotal: $100{,}480$

Hidden → output

weights: $10 \times 128 = 1{,}280$

biases: 10

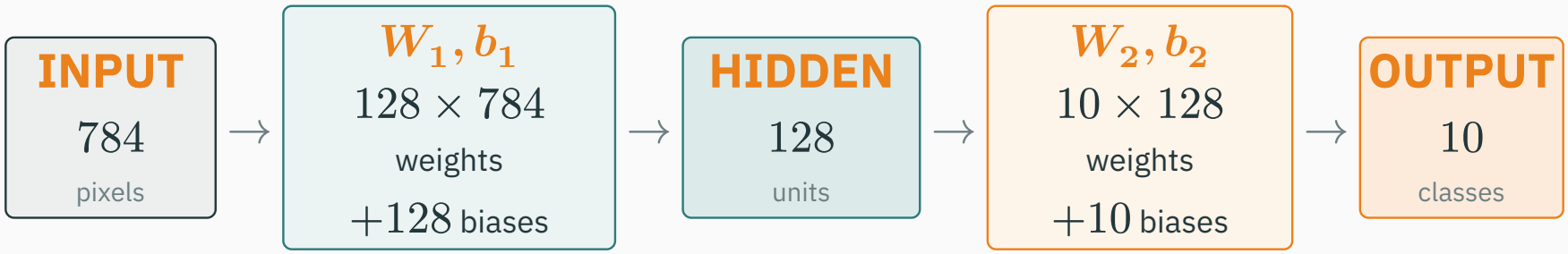
subtotal: $1{,}290$

Layer ℓ: $d_\ell d_{\ell-1}$ weights $+ d_\ell$ biases.

Count parameters by following the same network

D DERIVATION

Every connection has a **weight**; every receiving unit has a **bias**.



Input → hidden

weights: $128 \times 784 = 100{,}352$

biases: 128

subtotal: $100{,}480$

Hidden → output

weights: $10 \times 128 = 1{,}280$

biases: 10

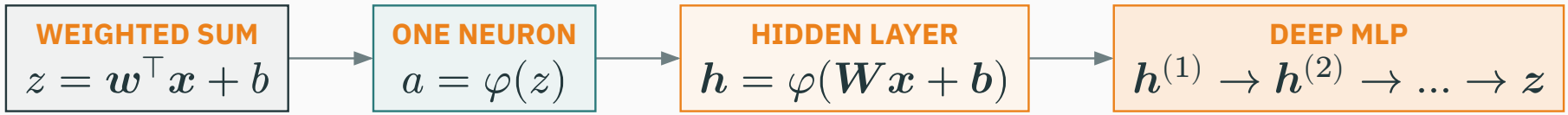
subtotal: $1{,}290$

Layer ℓ: $d_\ell d_{\ell-1}$ weights $+ d_\ell$ biases.

total $= 100{,}480 + 1{,}290 = 101{,}770$ **trainable parameters**

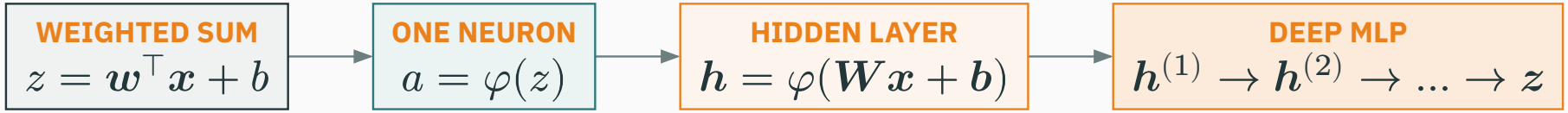
From one weighted sum to an MLP

V VISUAL



From one weighted sum to an MLP

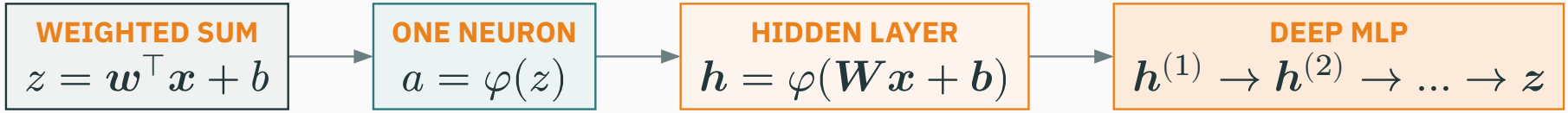
V VISUAL



Stage	Representation	What it adds
Weighted sum	$z = \boldsymbol{w}^\top \boldsymbol{x} + b$	One line or plane; captures a global linear trend.
One neuron	$a = \varphi(z)$	One nonlinear feature: a hinge, gate, or score.
Hidden layer	$\boldsymbol{h} = \varphi(\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b})$	Many learned features can bend or partition the input space.
Deep MLP	$\boldsymbol{h}^{(\ell)} = \varphi\big(\boldsymbol{W}_\ell \boldsymbol{h}^{(\ell-1)} + \boldsymbol{b}_\ell\big)$	Layers compose features into a hierarchical representation.

From one weighted sum to an MLP

V VISUAL



Stage	Representation	What it adds
Weighted sum	$z = w^\top x + b$	One line or plane; captures a global linear trend.
One neuron	$a = \varphi(z)$	One nonlinear feature: a hinge, gate, or score.
Hidden layer	$h = \varphi(Wx + b)$	Many learned features can bend or partition the input space.
Deep MLP	$h^{(\ell)} = \varphi(W_\ell h^{(\ell-1)} + b_\ell)$	Layers compose features into a hierarchical representation.

The key addition is not a new loss; it is a learned nonlinear representation.

The decisive change is the learned representation

Input x

$\downarrow$

learned nonlinear features $h^{(1)}, \ldots, h^{(L-1)}$

$\downarrow$

familiar Lecture 1 output model and loss

The decisive change is the learned representation

Input x

↓

learned nonlinear features $h^{(1)}, \ldots, h^{(L-1)}$

↓

familiar Lecture 1 output model and loss

The output loss can stay the same while hidden layers make the representation nonlinear.

We now have $x \to f_{\boldsymbol{\theta}}(\boldsymbol{x}) \to \mathcal{L}$.

Lecture 3: how do we compute $\nabla_{\boldsymbol{\theta}} \mathcal{L}$ efficiently?

computation graphs · the chain rule · backpropagation · automatic differentiation